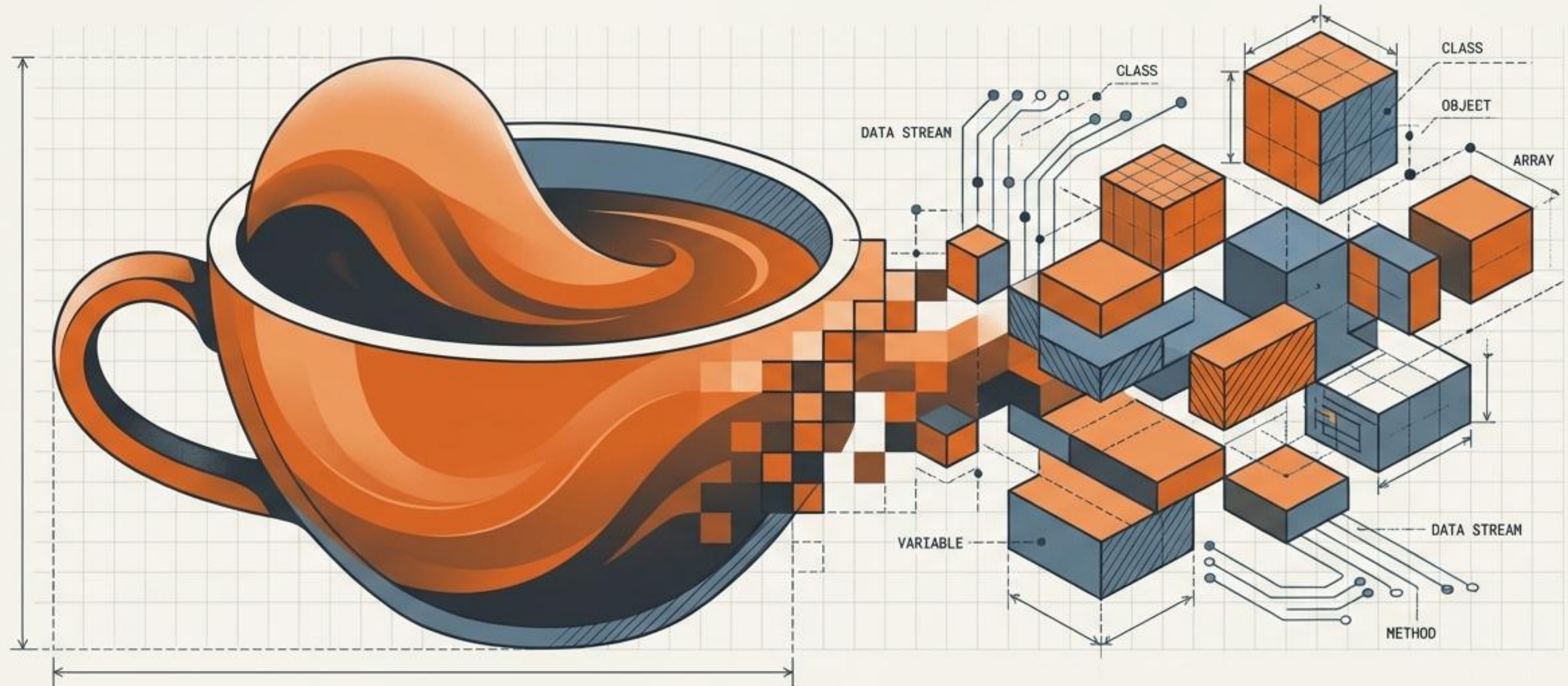


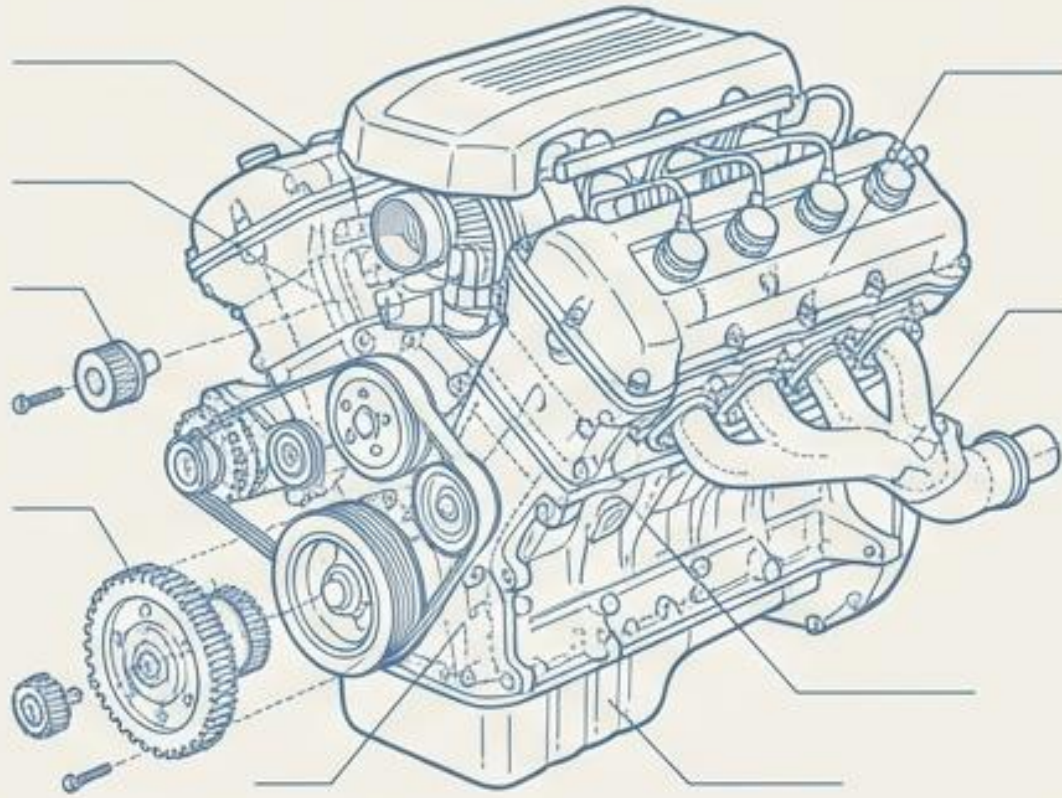
Java 프로그래밍의 첫걸음

기초 문법부터 데이터 처리의 핵심 원리까지



Day 1-5 Course Summary

환경 설정: 엔진과 대시보드



Java (JDK)

프로그램 실행을 위한 핵심 엔진

java.sun.com



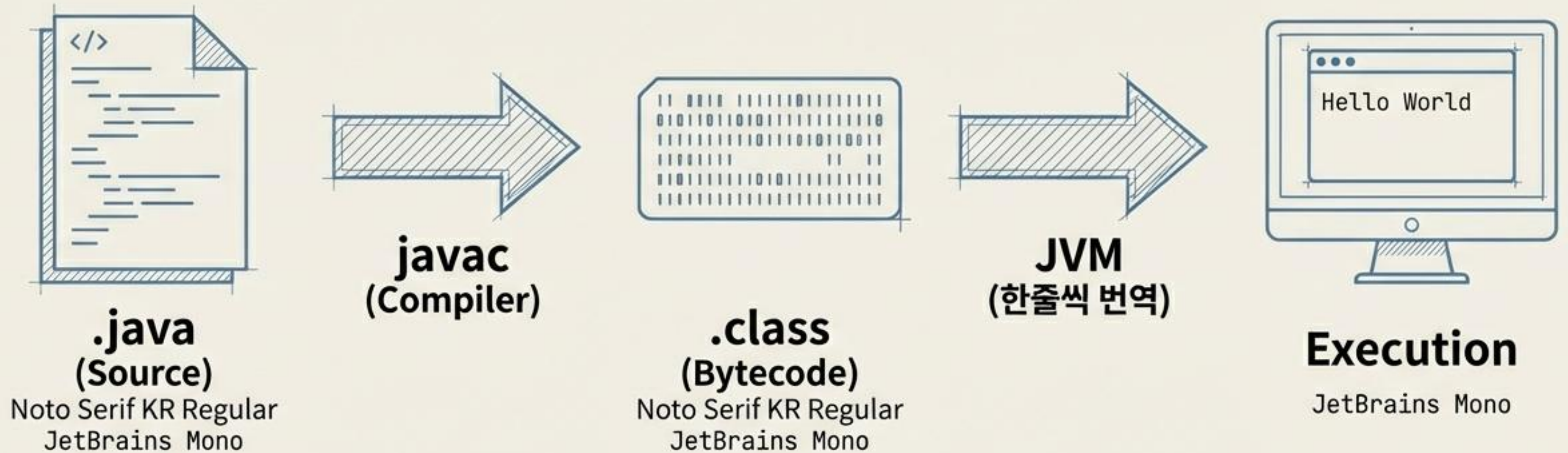
Eclipse

효율적인 코딩을 위한 통합 개발 환경

eclipse.org/downloads

Action Item: Java 설치 후 Eclipse 설치

실행 과정: 소스 코드에서 실행까지



JVM은 .class 파일을 읽어 한 줄씩 번역하여 실행합니다.

코드의 해부:

클래스와 메인 메소드

```
1 public class MainClass {  
2  
3     public static void main(String[] args) {  
4         // Program starts here  
5     }  
6 }
```

Class: 모든 자바 프로그램의 시작과 끝. 클래스명과 파일명은 반드시 동일해야 함.

Noto Serif KR Regular

Main Method: 프로그램의 진입점 (Entry Point).

Noto Serif KR Regular

반환값이 없음.

Noto Serif KR Regular

JVM은 .class 파일을 읽어 한 줄씩 번역하여 실행합니다.

명명 규칙과 식별자

PascalCase

클래스 (Class)

MainClass

대문자로 시작

camelCase

변수/메소드 (Variable/Method)

fileName

소문자로 시작

SCREAMING_SNAKE

상수 (Constant)

MAX_SIZE

모두 대문자

Hungarian Notation

참고용 (Style)

iCount

문법 아님

변수의 이해: 데이터 담기



`byte b = 10;`
`type variableName = value;`

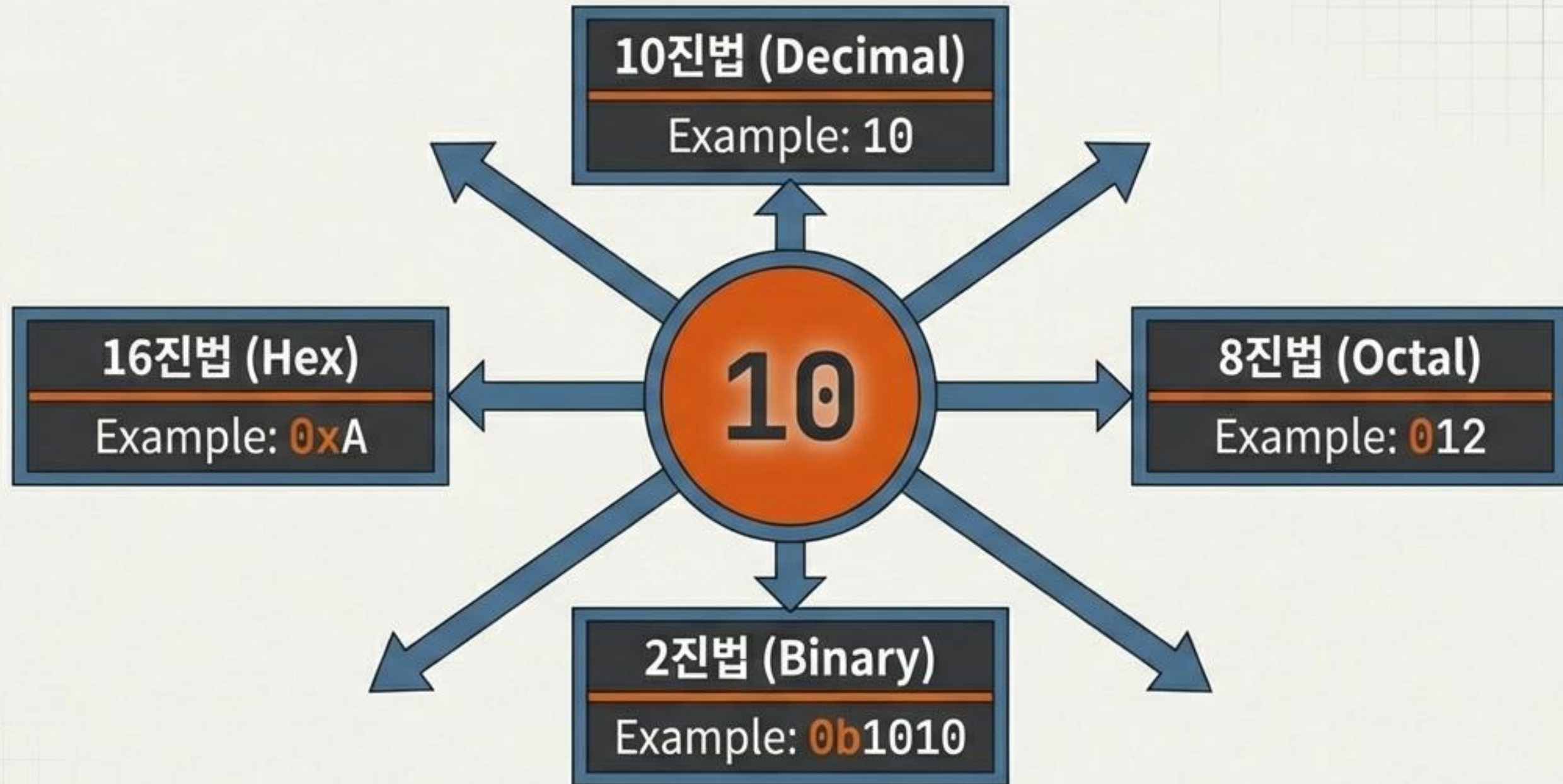
변수는 특정 데이터를 저장하는 메모리 공간의 이름입니다.

`byte b3 = b + b1;`

자바의 데이터형 지도

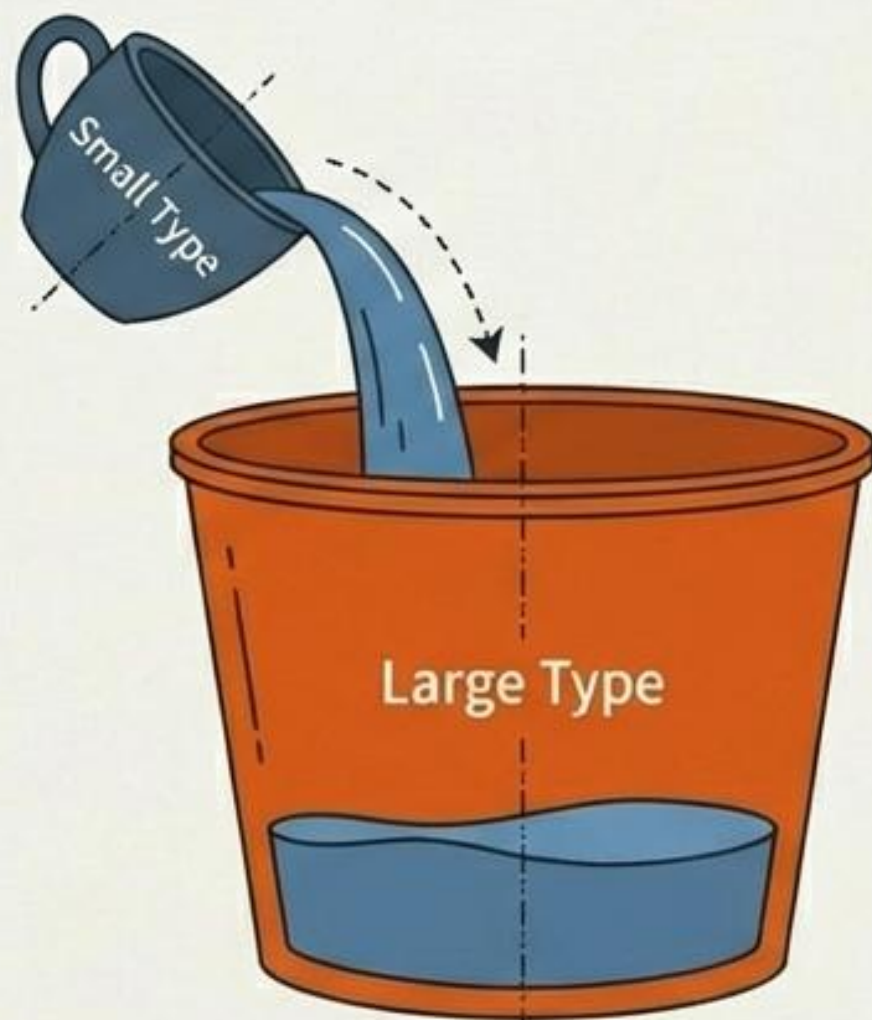


진법의 이해: 숫자의 표현

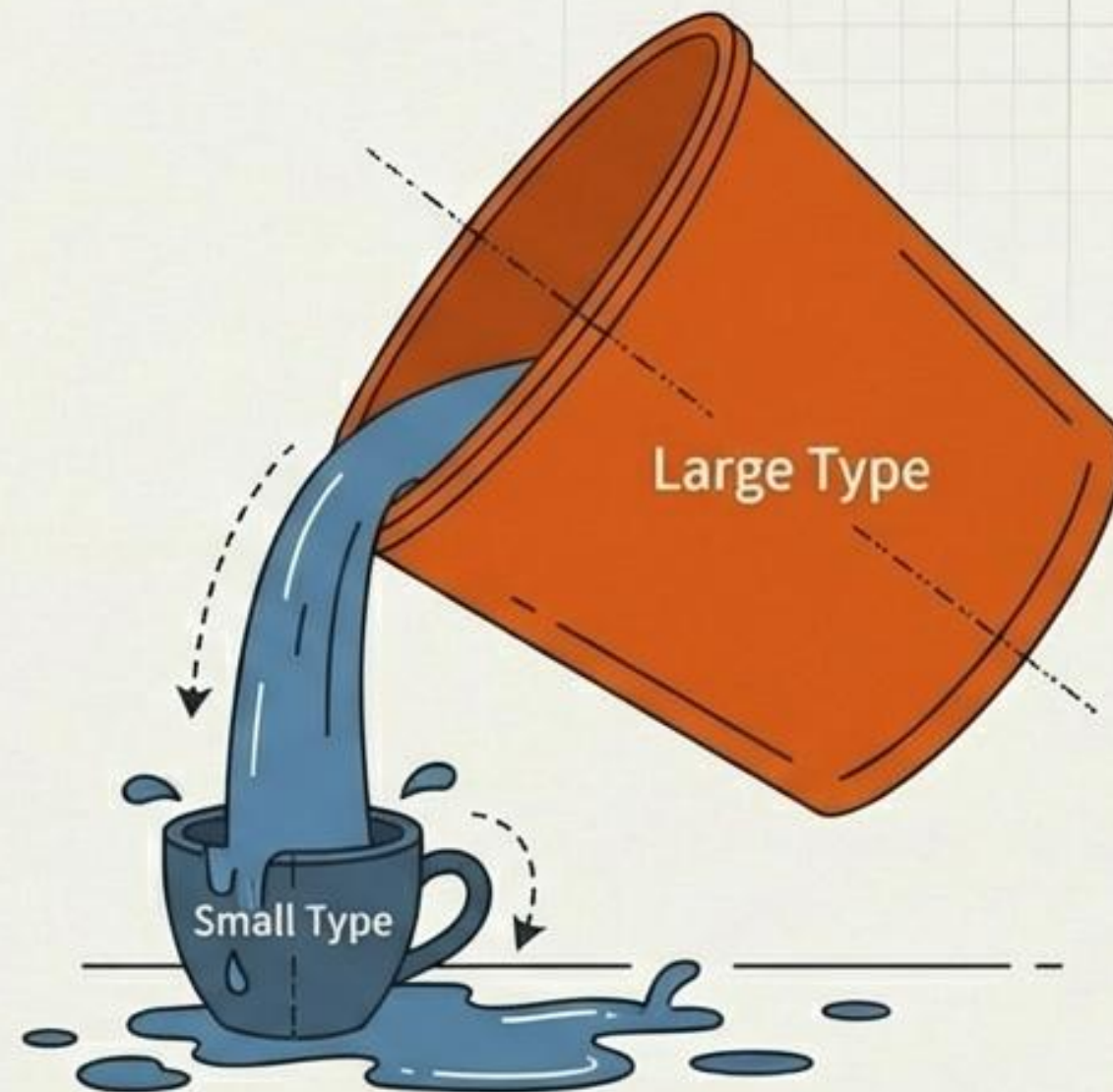


동일한 값을 컴퓨터에게 전달하는 다양한 방법

형변환(Casting)이란?



→ Safe →

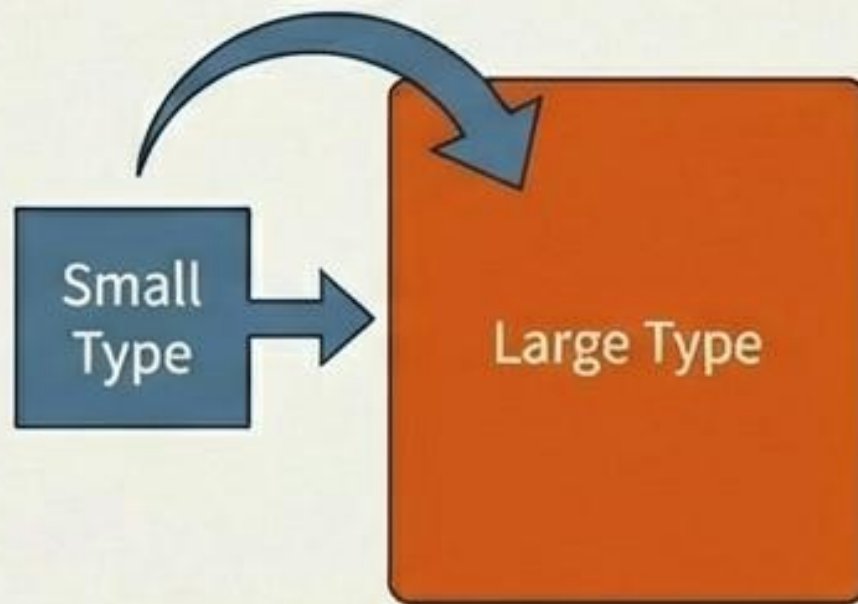


→ Data Loss / Overflow →

“형변환은 물을 큰 그릇과 작은 그릇 사이에서 옮기는 것과 같습니다.”

자동 vs 강제 형변환

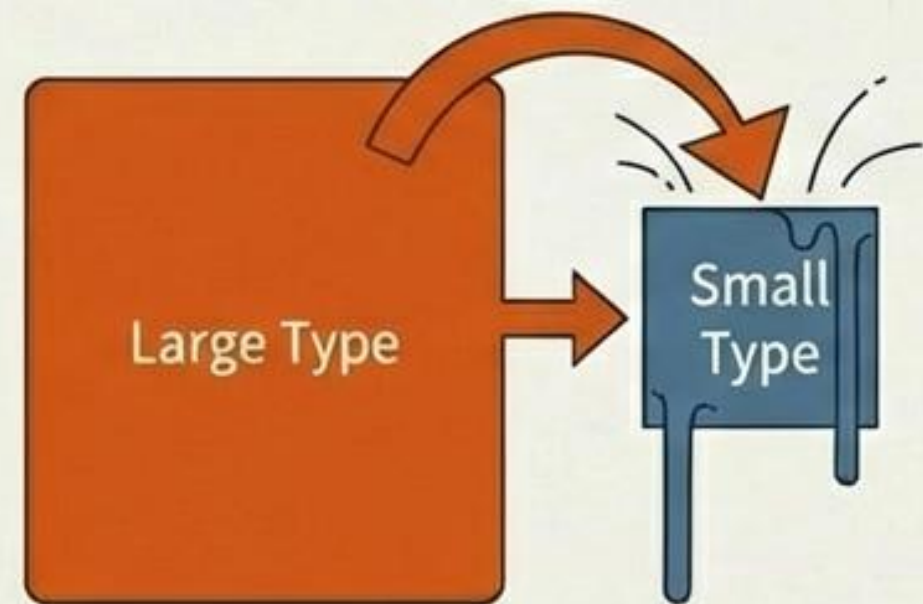
자동 형변환 (Automatic)



작은 타입 -> 큰 타입

✓ 데이터 손실 없음

강제 형변환 (Explicit)



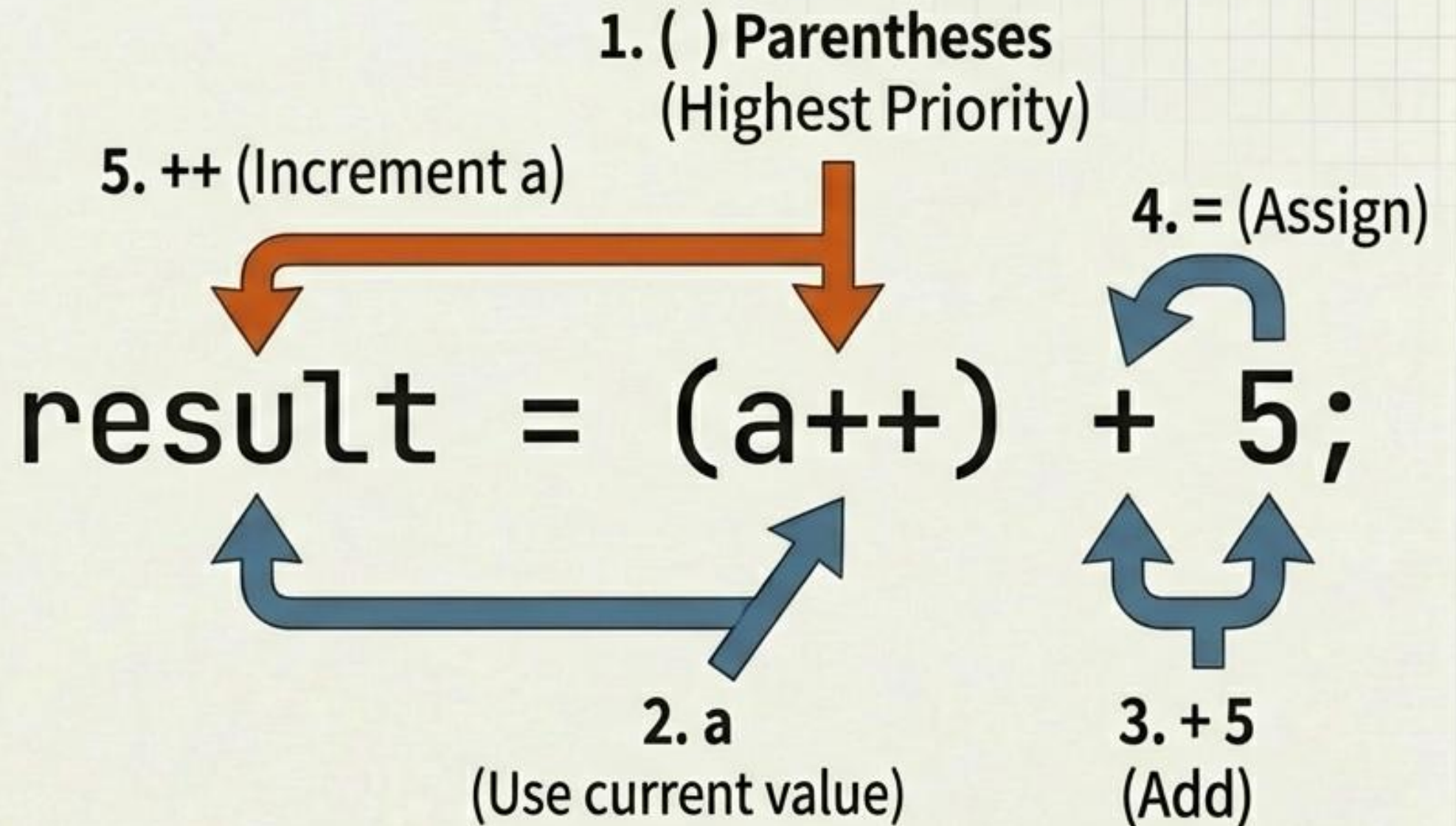
큰 타입 -> 작은 타입
(int)variable

⚠ 데이터 손실 발생 가능

단항 연산자와 우선순위

- ++ (Increment)
 - Noto Serif KR Regular
- -- (Decrement)
 - Noto Serif KR Regular
- ! (Logical Not)
 - Noto Serif KR Regular

최우선 순위는 괄호 () 입니다.



이항 및 삼항 연산자

산술

Noto Serif KR Regular

+ - * / %

비교

Noto Serif KR Regular

> < >= <= == !=

논리

Noto Serif KR Regular

&& ||

대입

Noto Serif KR Regular

= += -=

비트/쉬프트

Noto Serif KR Regular

& | ^ << >>

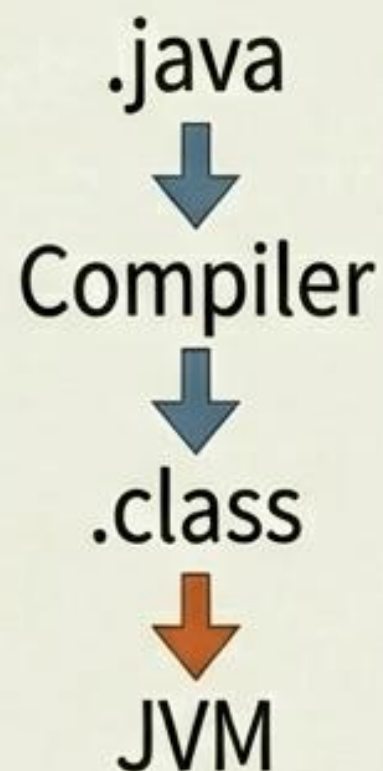
삼항 연산자 (Ternary)

Noto Serif KR Regular

(cond) ? true : false

핵심 요약: 자바 5일 완성

실행 (Execution)



타입 (Types)

Byte
Int
Double
Boolean
String

변수 (Variables)

camelCase

type name
= value

형변환 (Casting)

Auto
(Small->Big)

Forced
(Big->Small)


연산자 (Operators)

Precedence:
()
Unary
Binary
Ternary

주의사항 및 팁

Java Programming Checklist



자바는 대소문자를 엄격히 구분합니다 (Case Sensitive).



저장 시 반드시 .java 확장자 사용.



주석(Comments): `//` 한 줄, `/* */` 여러 줄.

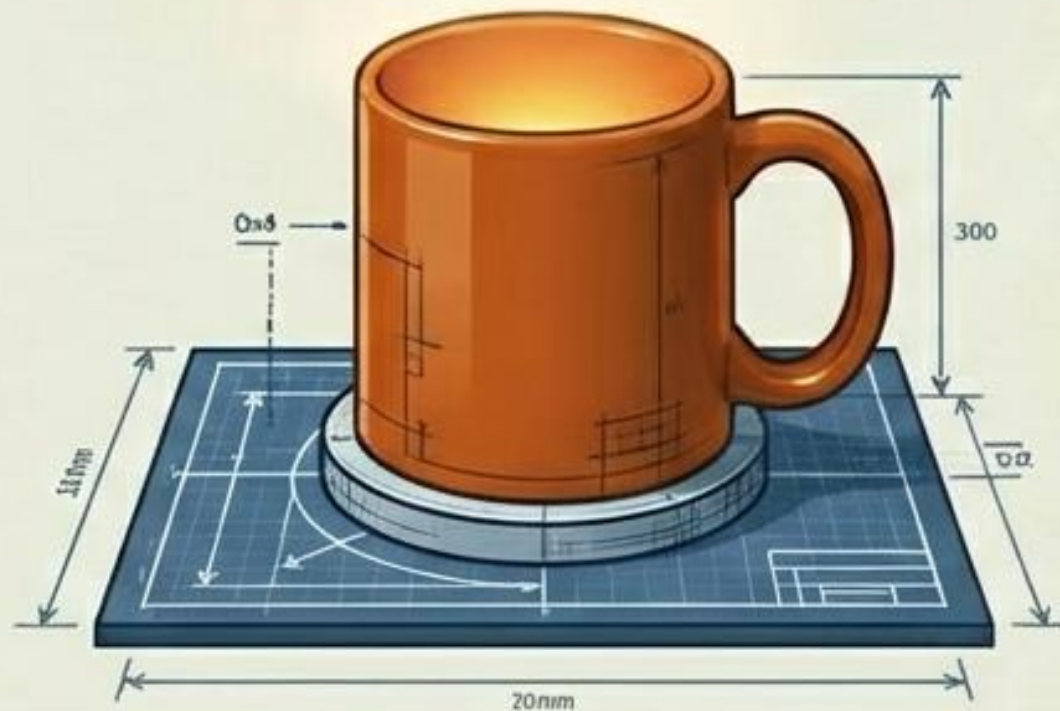


주석은 컴파일 시 실행되지 않고 제외됩니다.

Java 정복을 위한 준비 완료

변수, 연산자, 그리고 논리

기초가 튼튼하면 어떤 코드도 두렵지 않습니다.
연습을 통해 문법을 익히세요.



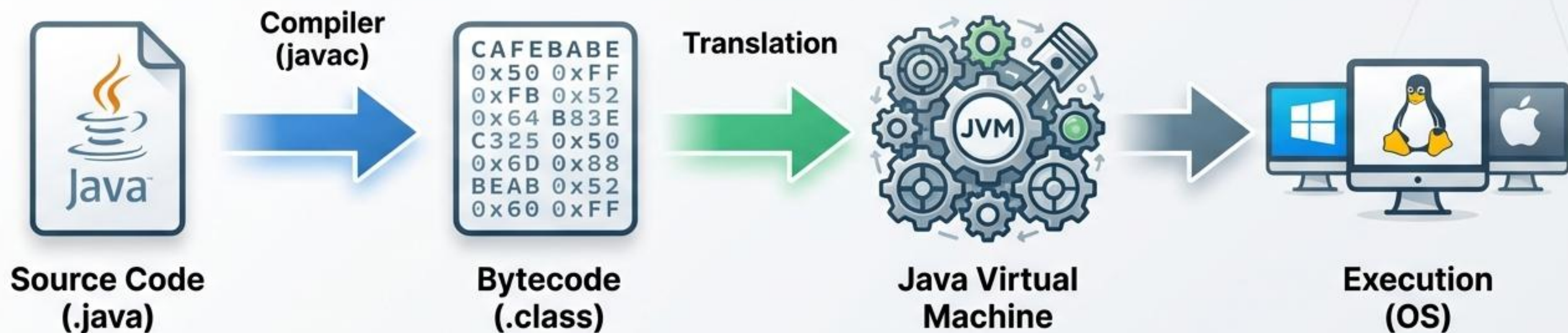
Java: 문법을 넘어 아키텍처로

변수부터 객체지향, 그리고 모던 스트림 API까지의 여정



Architecture of a Backend Developer

자바의 구동 원리: Write Once, Run Anywhere



Key Points

-  **독립성:** OS에 종속되지 않는 실행 환경
-  **실행 흐름:** 소스(.java) → 바이트코드(.class) → JVM
-  **진입점:** `public static void main(String[] args)`

데이터를 담는 그릇: 변수와 자료형

기본형 (Primitive Types)



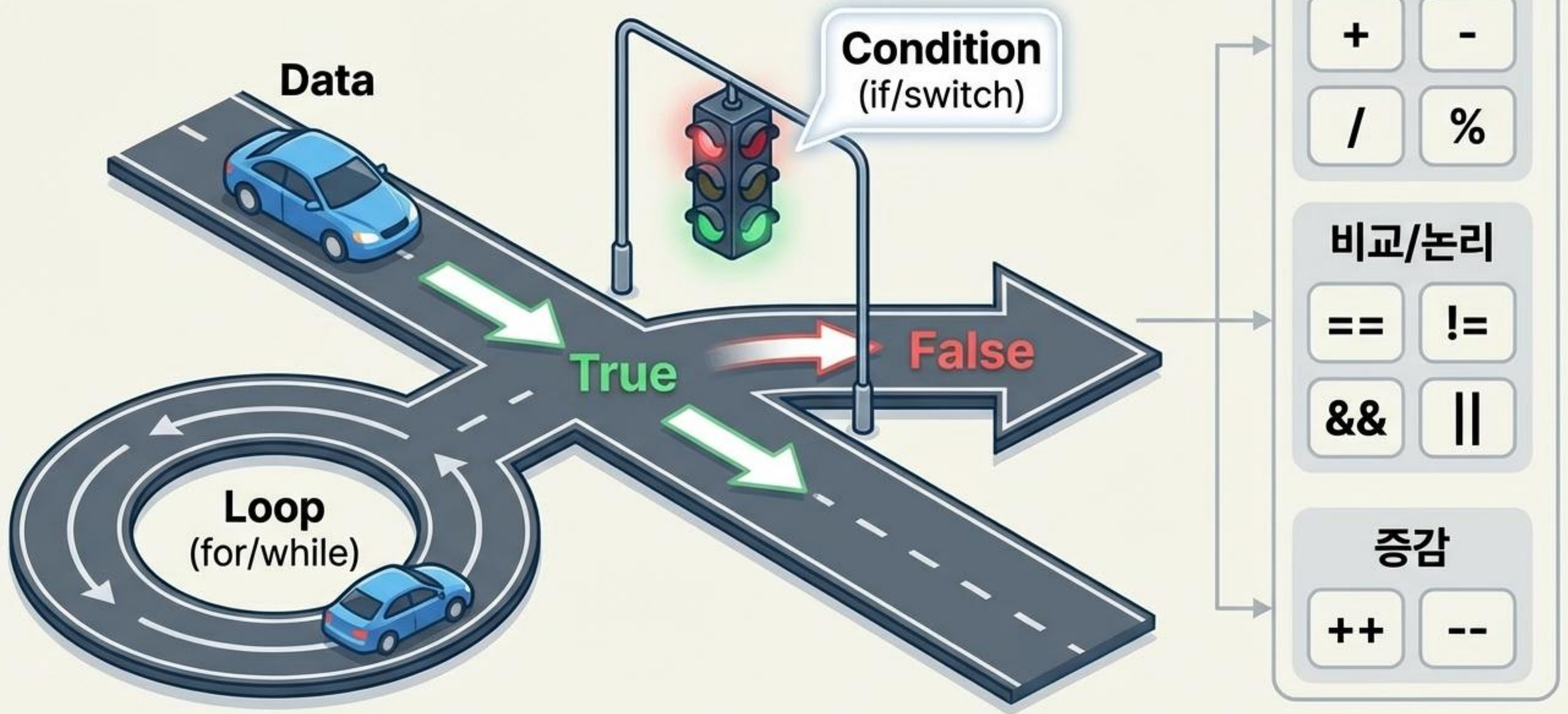
참조형 (Reference Types)



형변환 (Casting)

작은 컵의 물을 큰 양동이에 붓는 것은 안전하지만(자동), 큰 양동이의 물을 작은 컵에 부을 때는 넘칠 수다(강제).

논리의 흐름 제어: 연산자와 제어문



데이터의 정리 정돈: 배열 (Array)



```
int[] numbers = new int[5];  
numbers[0] = 10;
```

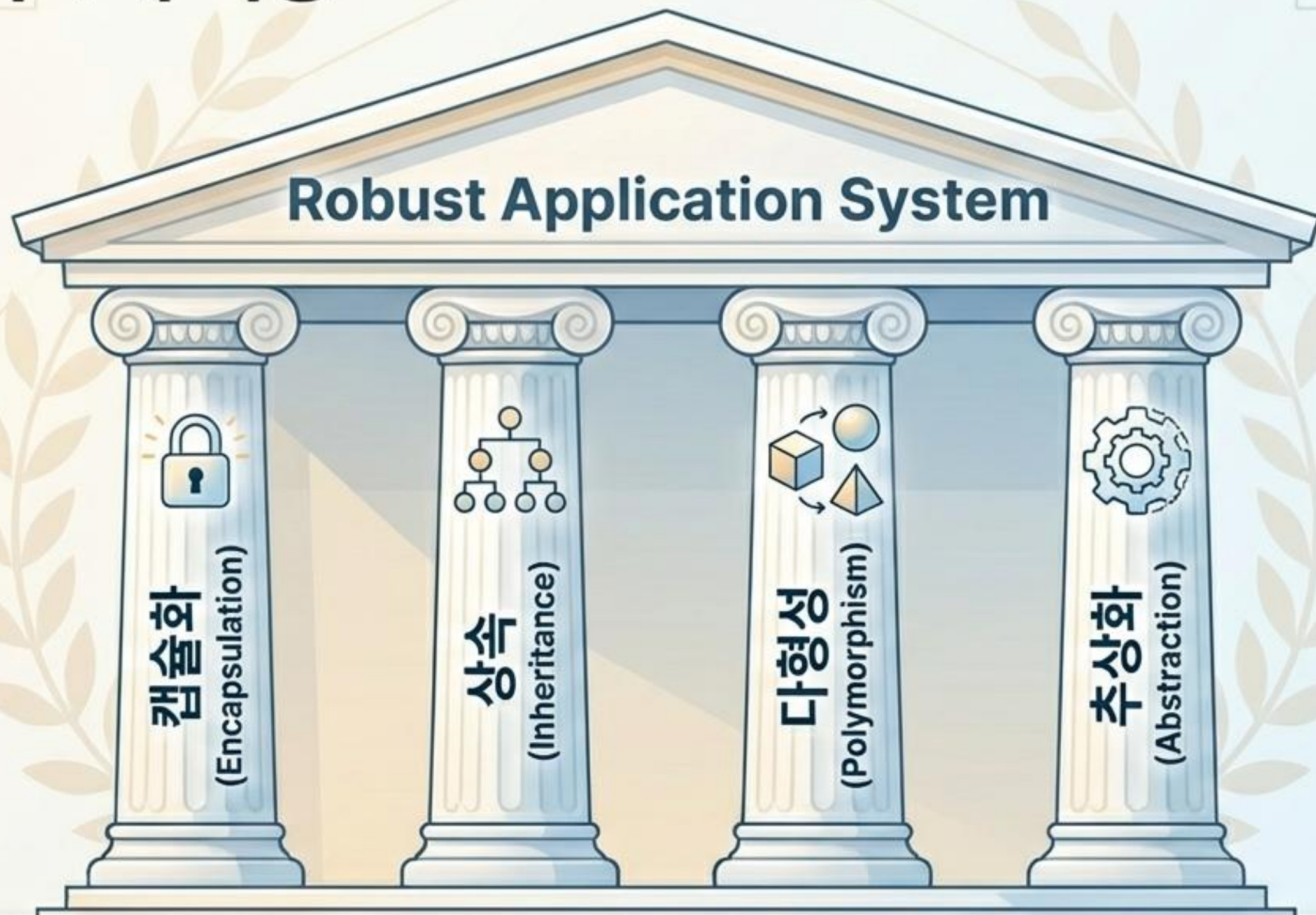
- **인덱스(Index)** : 0부터 시작하는 주소 체계
- **특징** : 생성 시 크기가 고정됨

설계도와 실체: 클래스와 객체



- **클래스** : 객체를 생성하기 위한 틀 (필드 + 메서드)
- **객체** : 메모리에 실체화된 결과물
- **생성자** : 객체를 초기화하는 과정

객체지향의 4대 기둥



자바가 대규모 시스템 구축에 강력한 이유는 이 4가지 구조적 특징 때문입니다.

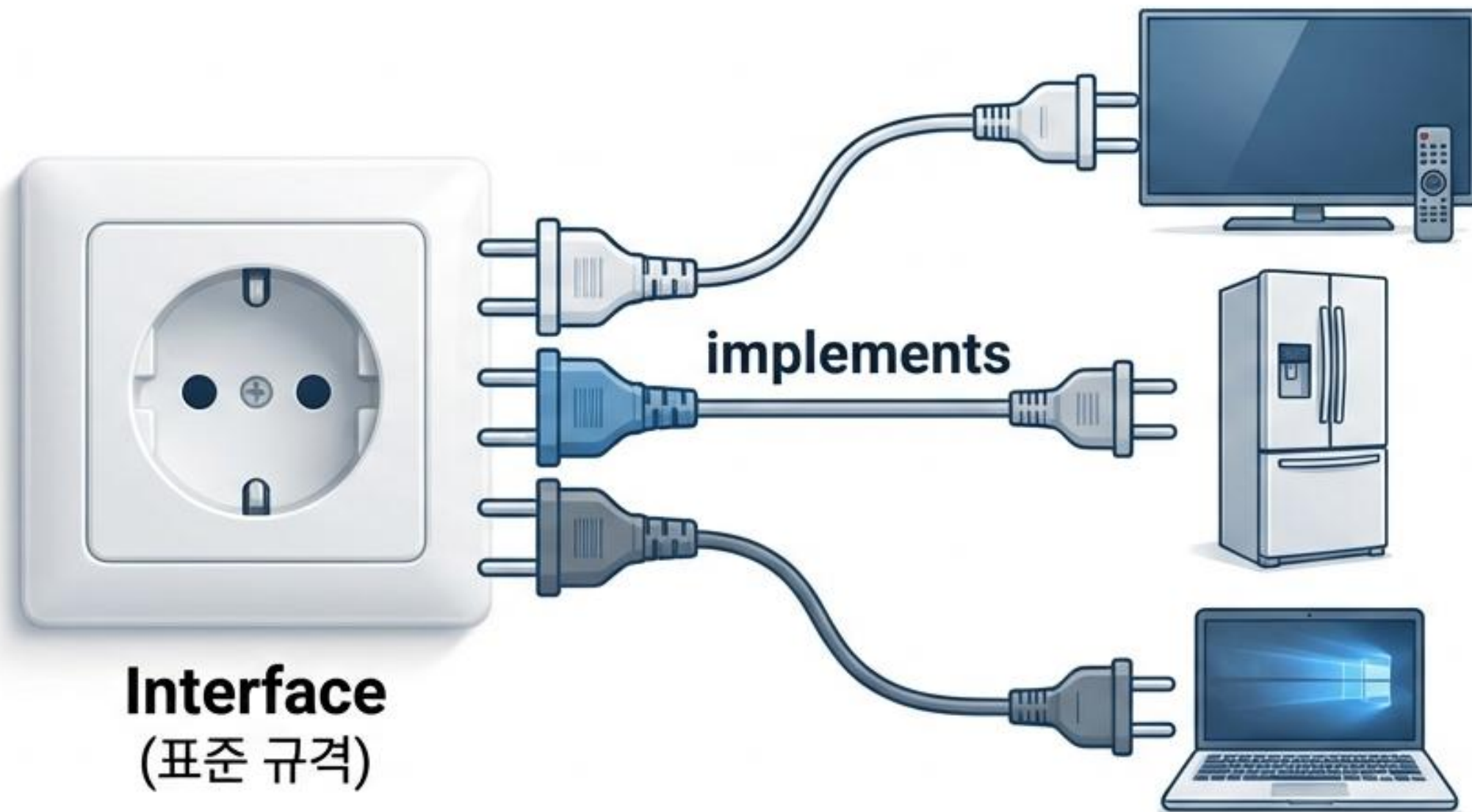
유연한 구조의 핵심: 상속과 다형성



```
Animal animal = new Dog();  
  
animal.sound() → 멍멍!  
@Override (재정의)
```

부모 타입으로 자식 객체를 참조하여 유연한 설계를 가능하게 합니다.

약속과 표준화: 인터페이스 (Interface)



- ⚙️ **기능의 표준화:** 내부 구현은 달라도 사용법은 동일.
- ↕️ **다중 구현:** 하나의 객체가 여러 인터페이스(규격)를 가질 수 있음.
- 🤝 **협업의 도구:** 인터페이스만 알면 소통 가능.

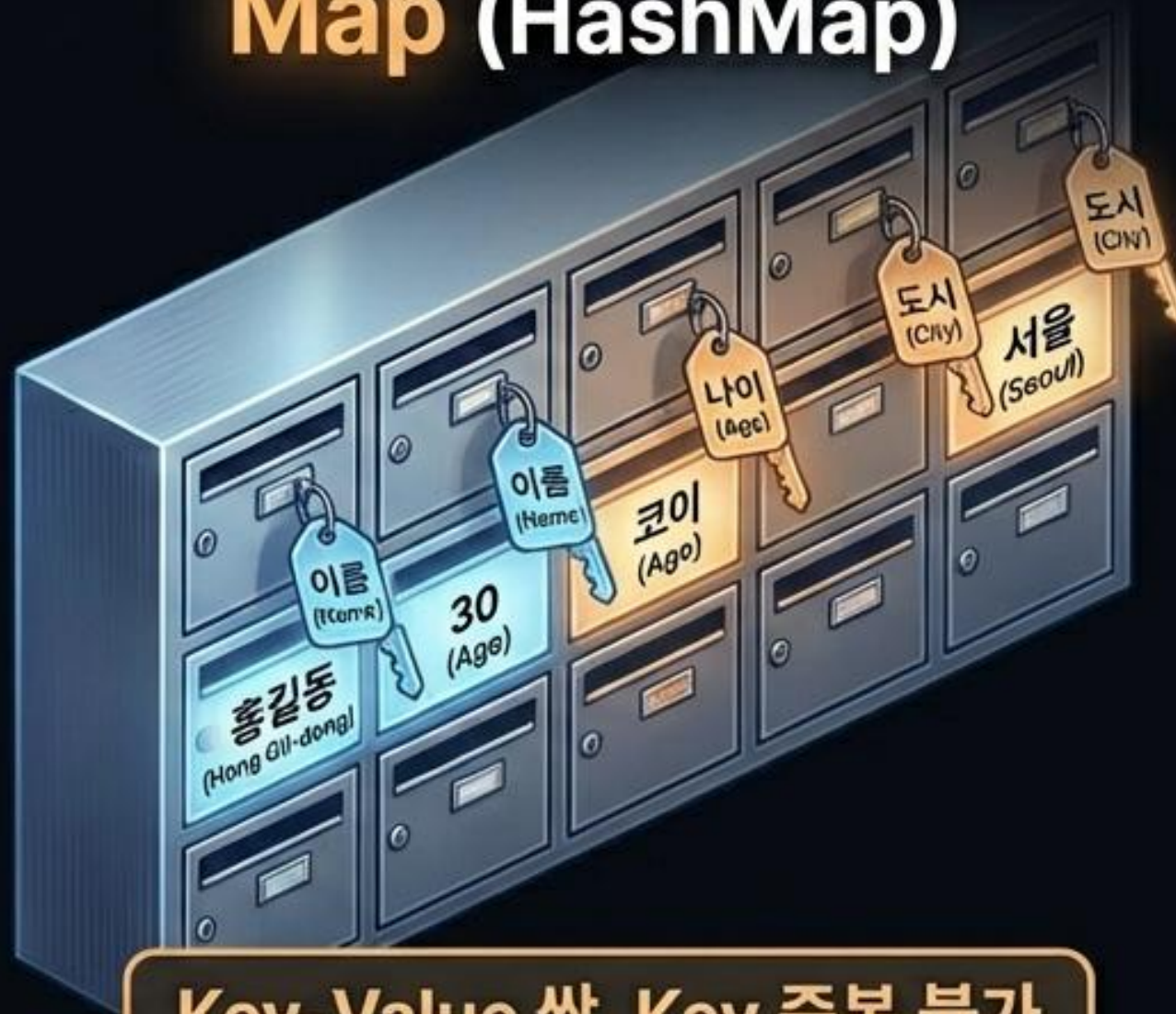
스마트한 데이터 관리: 컬렉션 프레임워크

List (ArrayList)



순서 있음 (Ordered), 중복 허용

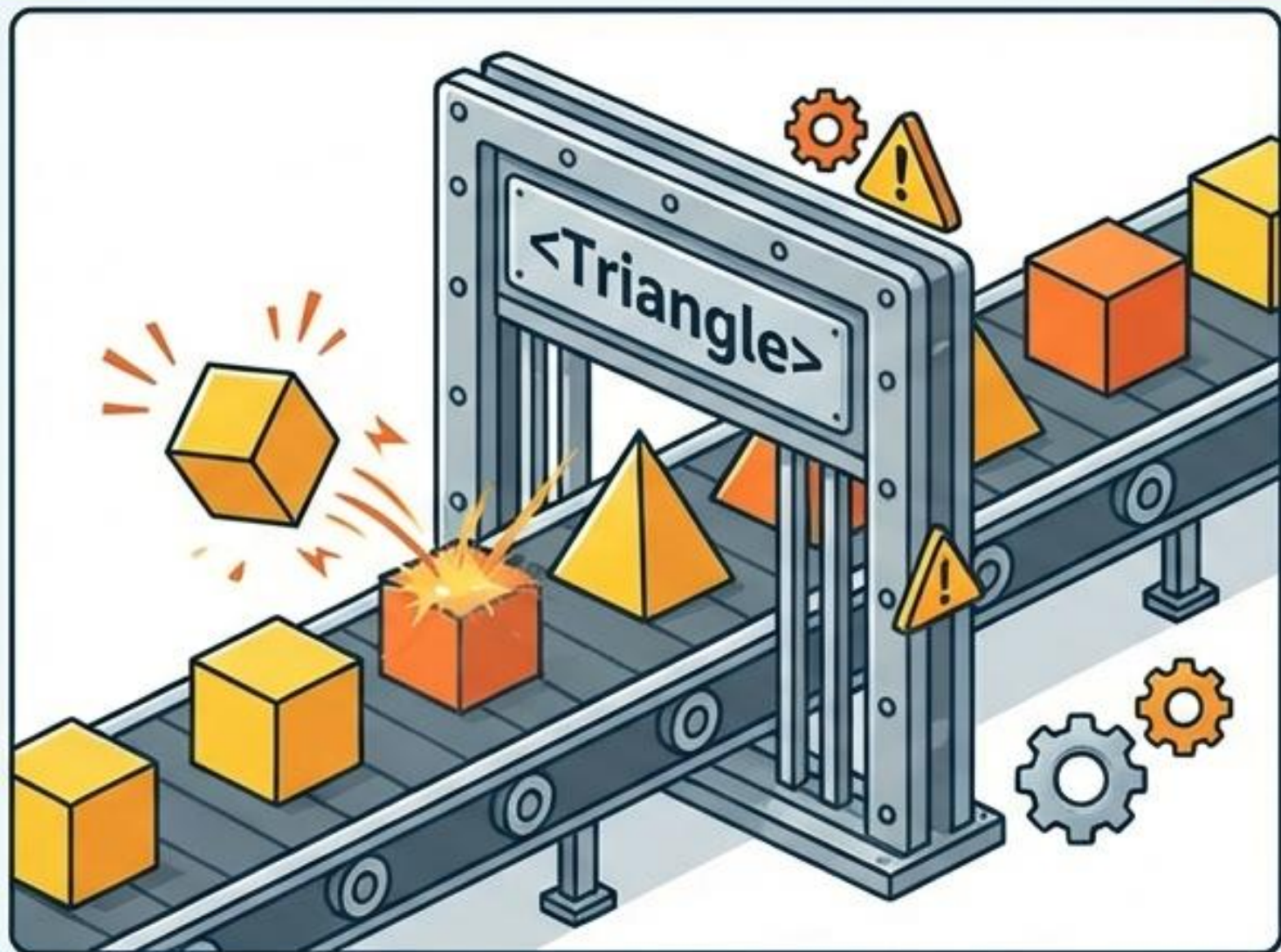
Map (HashMap)



Key-Value 쌍, Key 중복 불가

배열의 크기 제약을 넘어, 데이터를 탄력적이고 효율적으로 관리합니다.

안전장치: 제네릭(Generics)과 예외처리



Compile Time Safety (타입 체크)

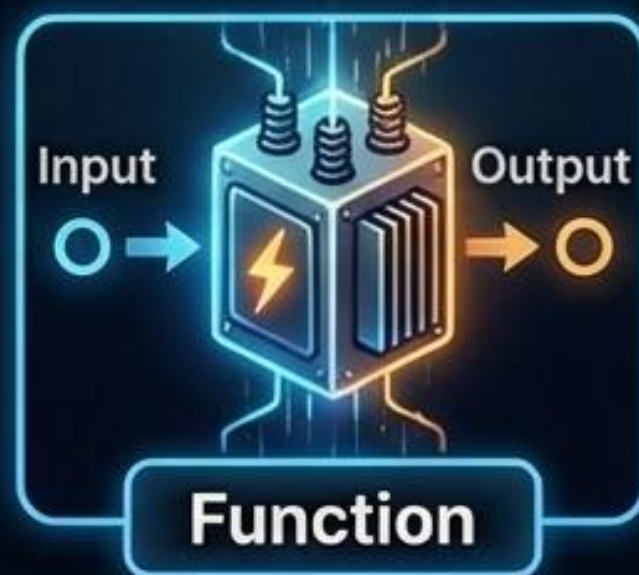
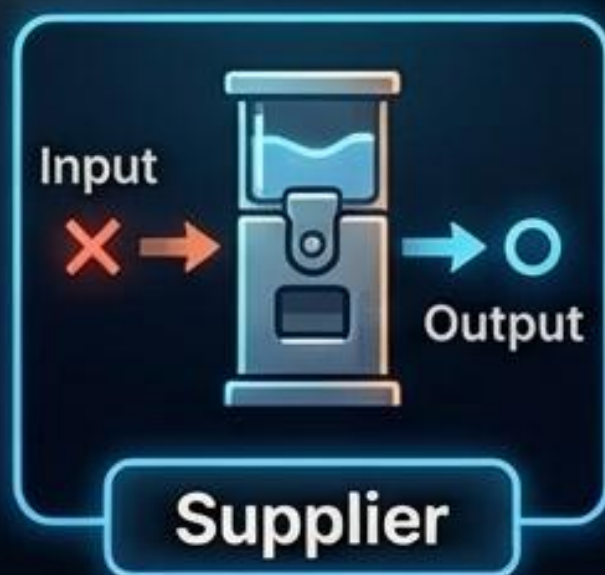
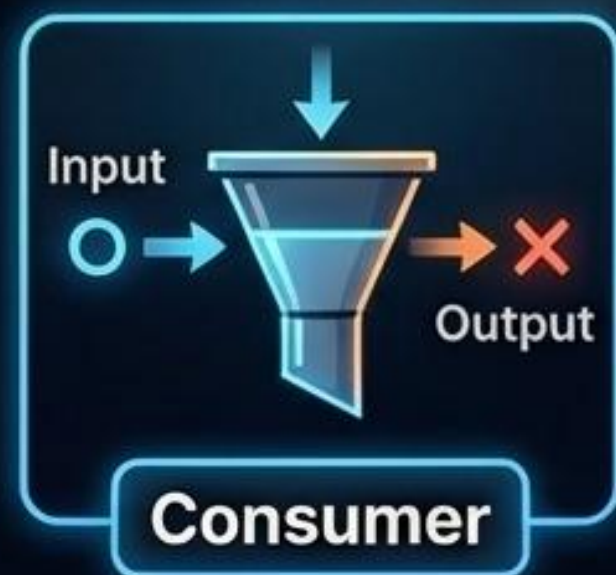
`List<String>`



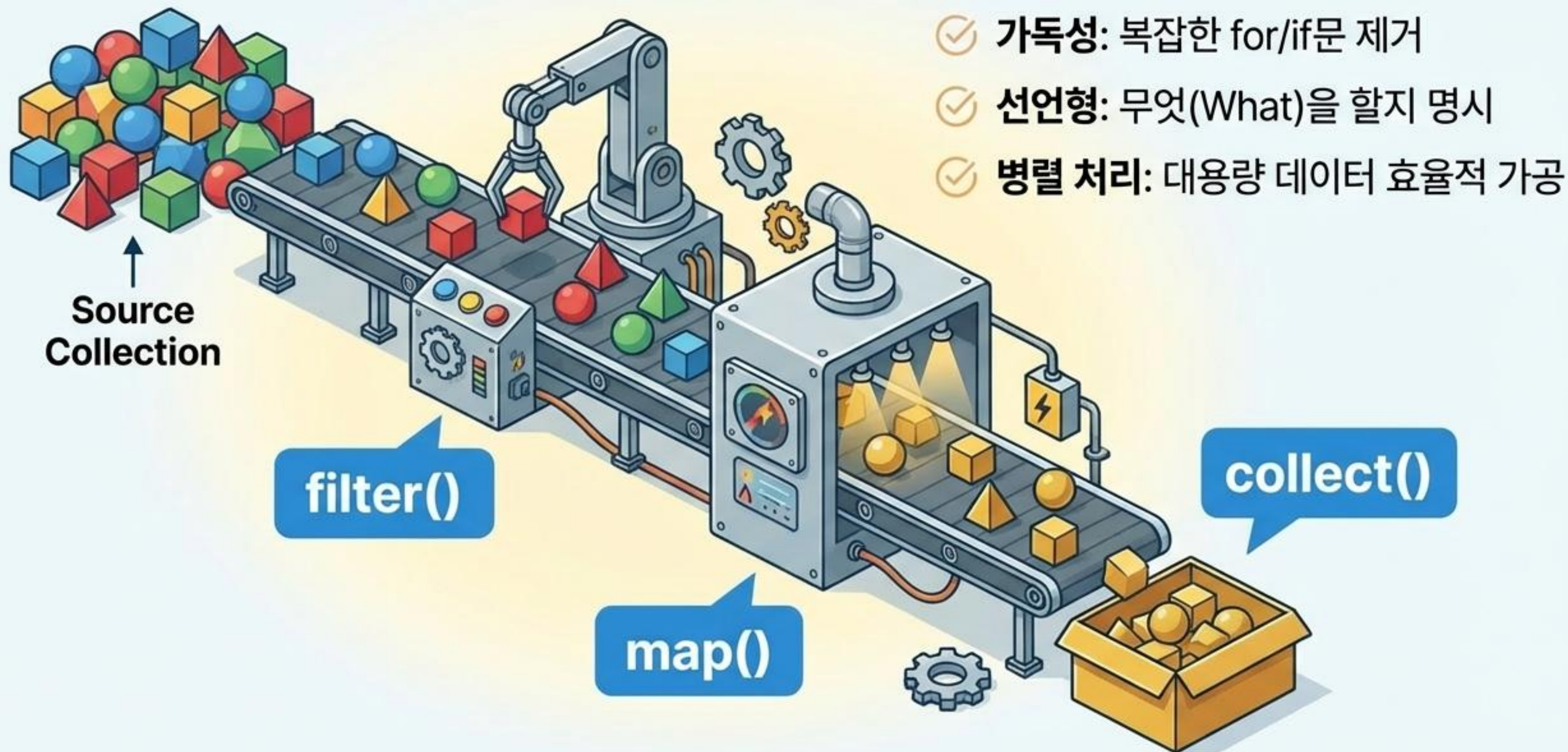
Runtime Safety (try-catch 예외 처리)

```
try { ...  
} catch (Exception e) { ... }
```

간결함의 미학: 람다와 함수형 인터페이스



데이터 처리 파이프라인: Stream API



Null의 공포로부터 해방: Optional

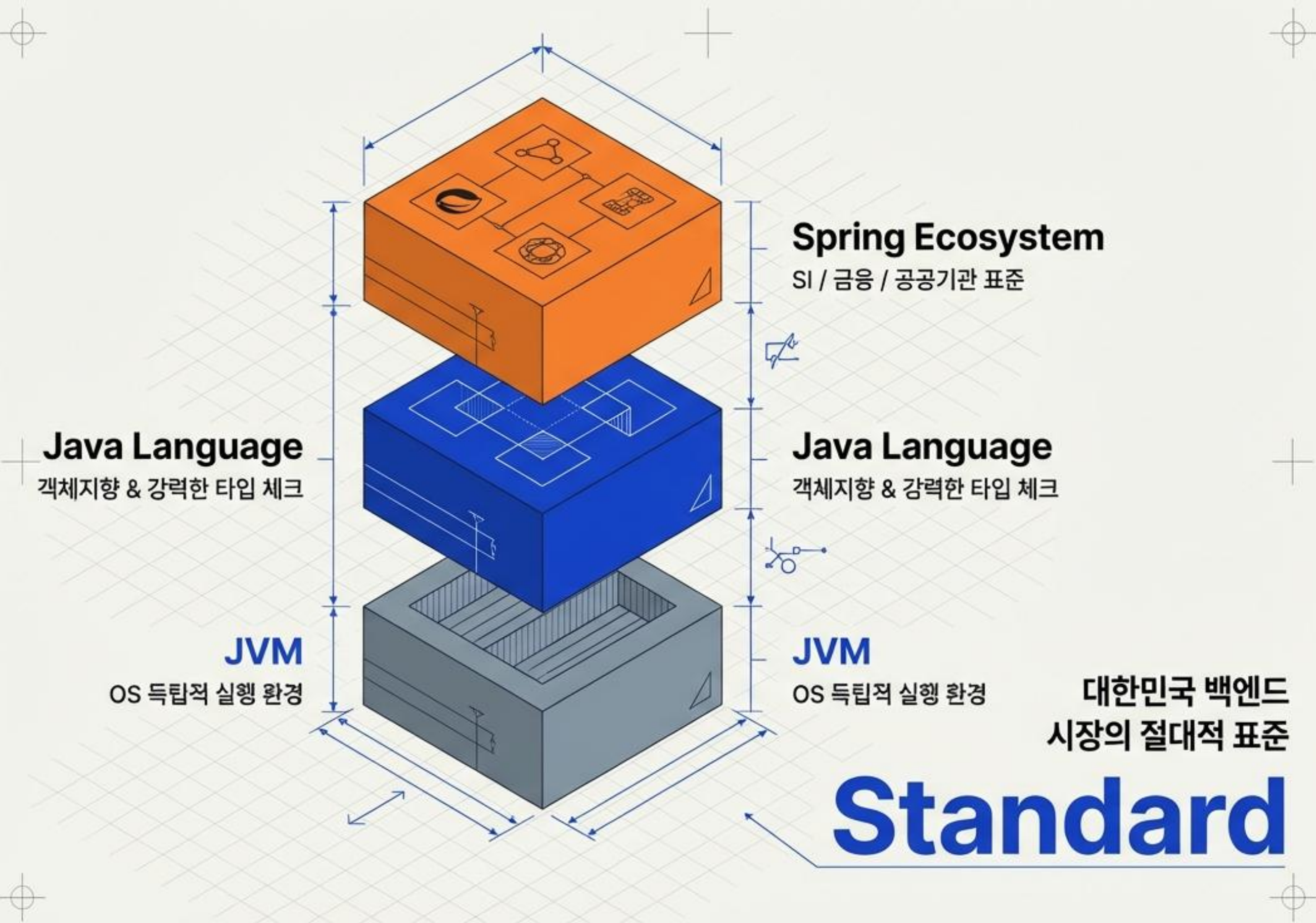


NullPointerException을 방지하고, 값이 있거나 없는 상태를 안전하게 다루는 래퍼(Wrapper) 클래스.

백엔드 개발자를 위한 핵심 요약 (Mastery Roadmap)

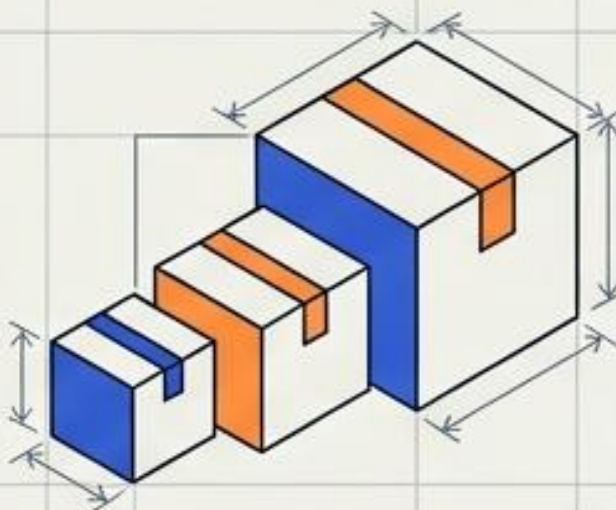


“자바는 단순한 언어가 아니라, 견고한 시스템을 짓는 도구입니다.”
Start coding with Interface & Stream today.



자바는 단순한 언어가 아닙니다. 거대한 엔터프라이즈 시스템을 지탱하는 가장 안정적인 뿌리입니다.

자바는 단순한 언어가 아닙니다. 거대한 엔터프라이즈 시스템을 지탱하는 가장 안정적인 뿌리입니다.



Variables (변수)

데이터를 담는 그릇

int (정수) double (실수)
String (문자열)

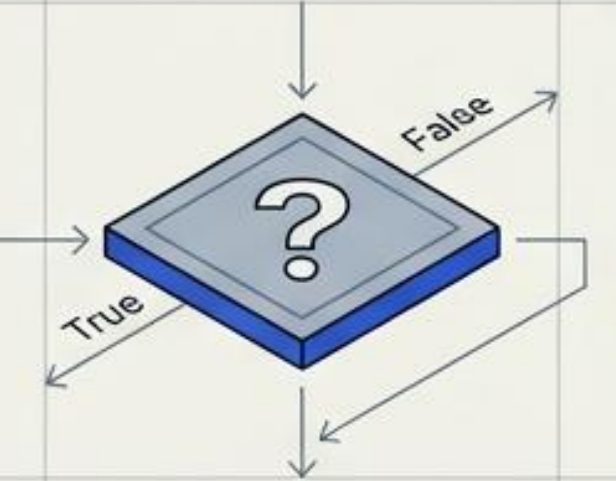


Operators (연산자)

데이터를 가공하는 도구

+ - % ==
== ++

도구
(Tools)



Control Flow (제어문)

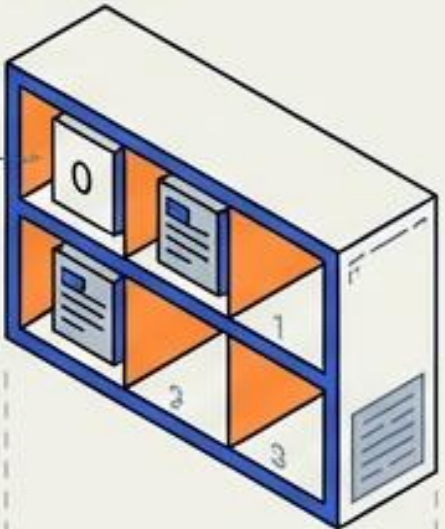
흐름을 제어하는 나침반

if → for → while

Arrays (배열)

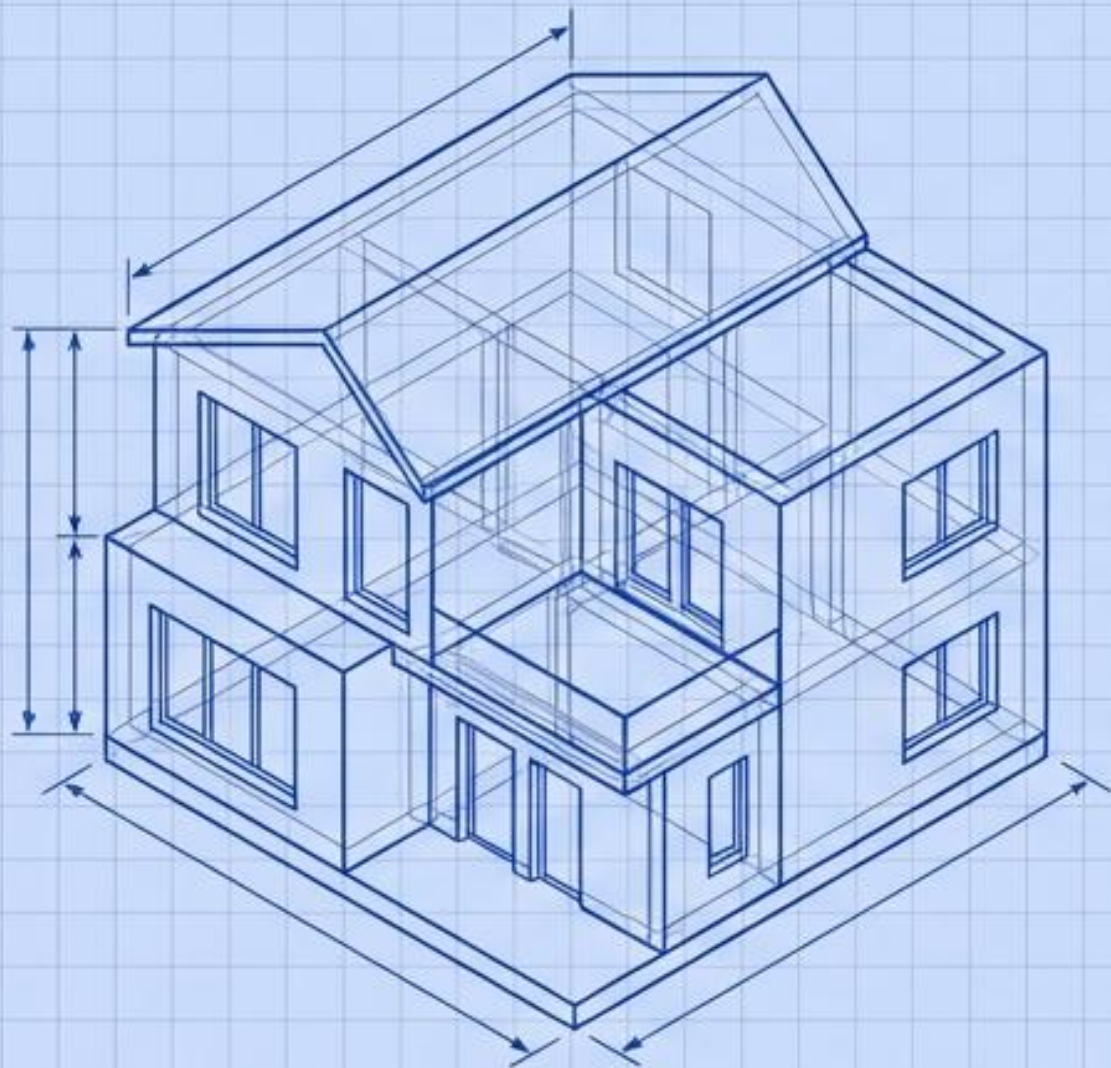
같은 종류를 담는 고정 수납장

Index 0 Fixed Size



문법은 외우는 것이 아니라, 필요할 때 꺼내 쓰는 도구입니다.

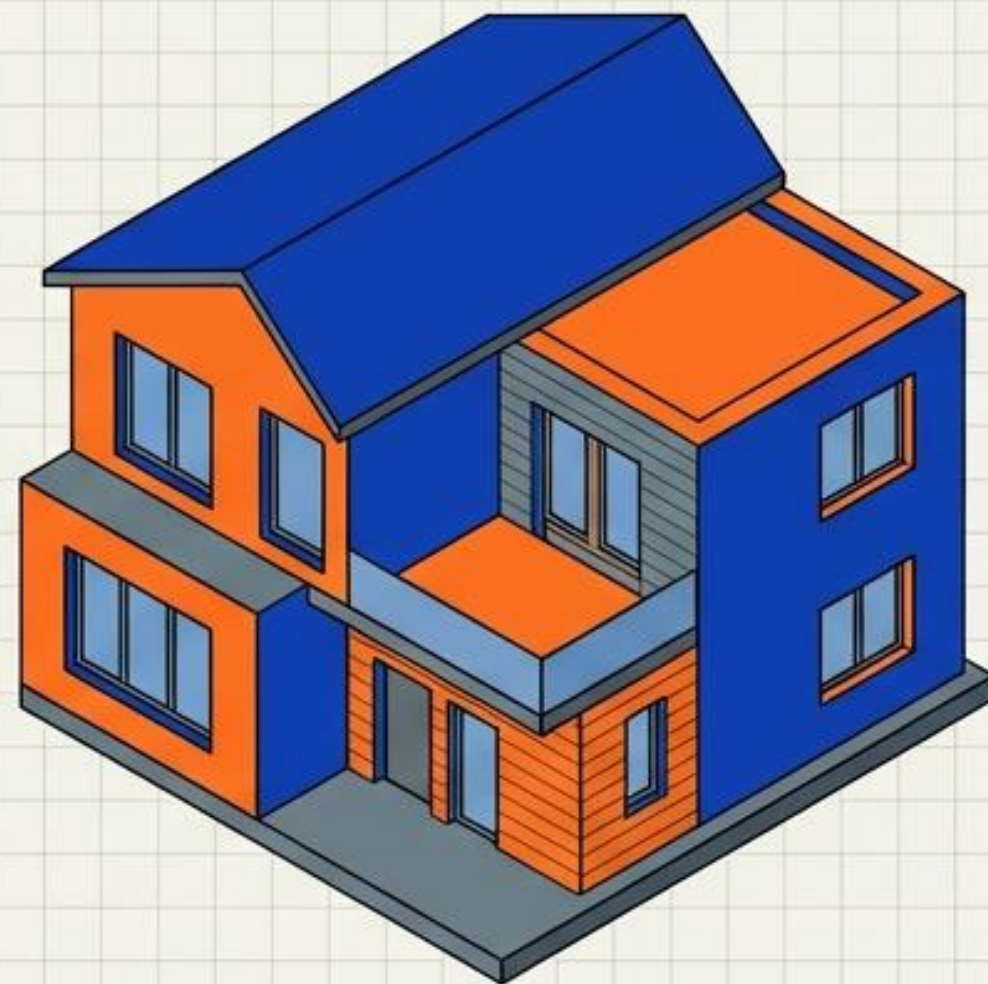
Class (설계도)



```
class House { .. }
```

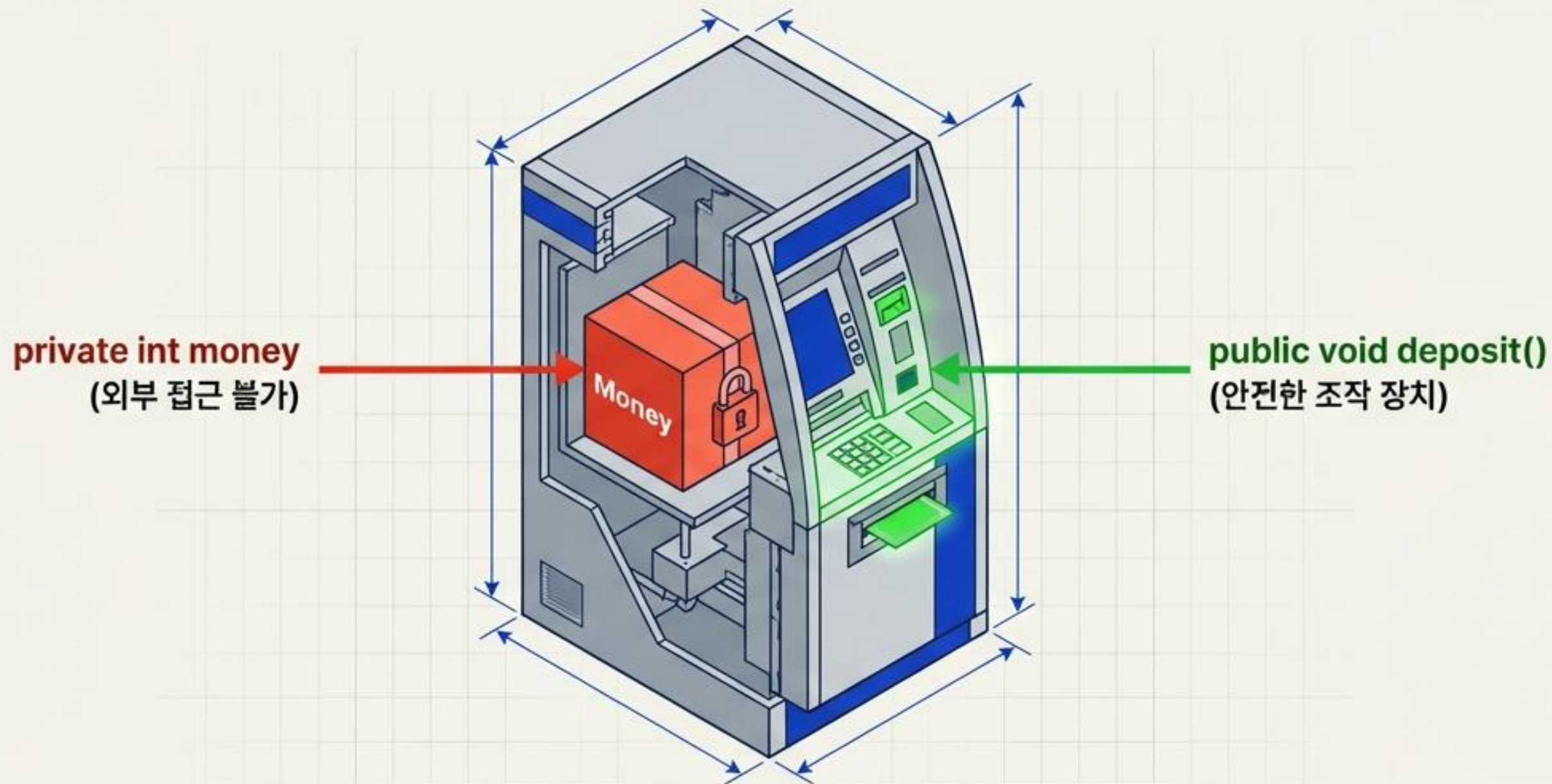
new

Object (실체/인스턴스)



```
new House();
```

설계도(Class)는 하나지만, new 키워드를 통해 수많은 실체(Object)를 메모리에 만들어냅니다.

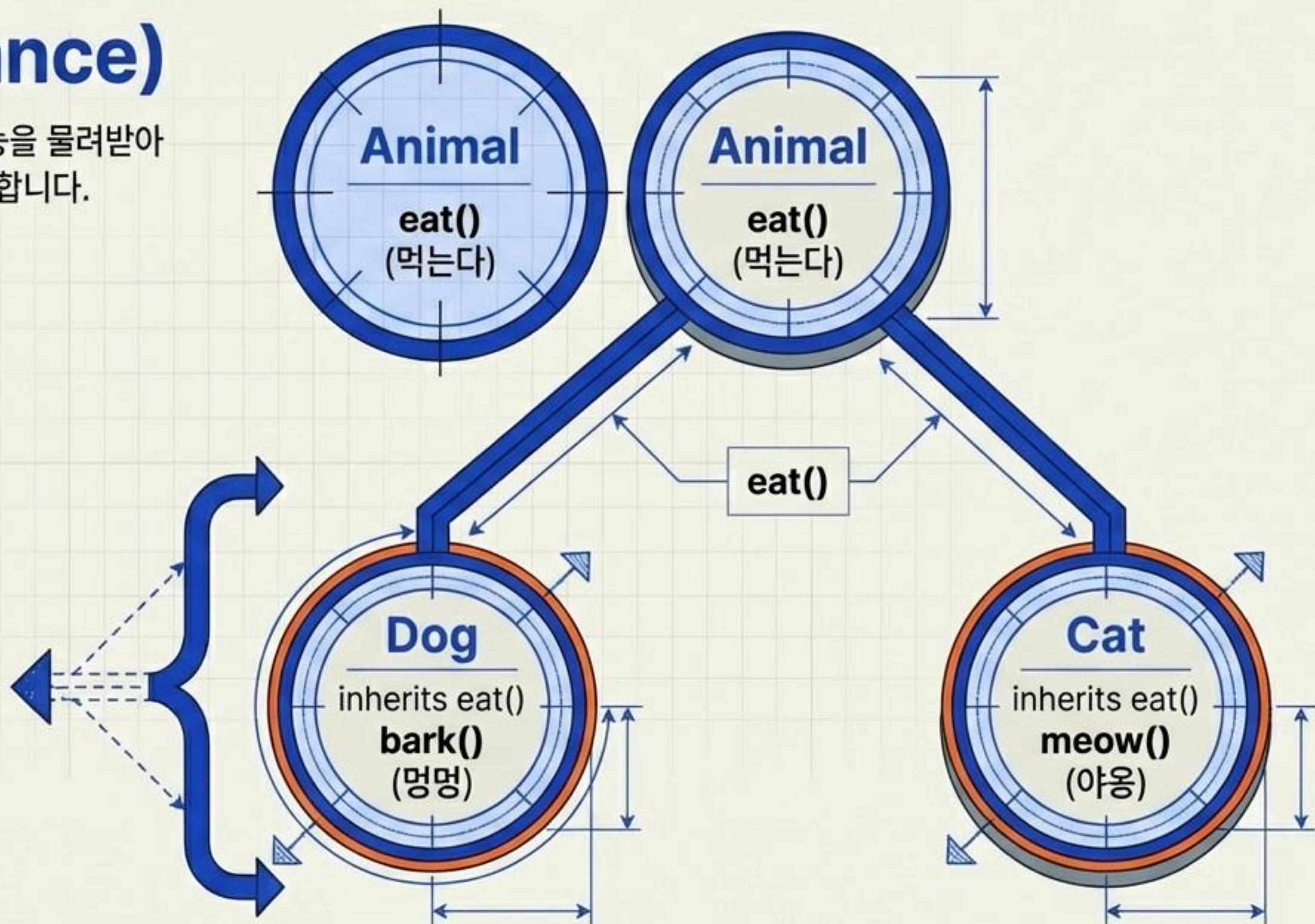


private (데이터 보호) + **public Getter/Setter** (무결성) = **캡슐화 (Encapsulation)**

ATM 내부의 돈을 직접 가져갈 수는 없습니다. 반드시 제공된 입출금 버튼을 통해서만 접근해야 합니다.

상속 (Inheritance)

extends 키워드 사용. 부모의 기능을 물려받아
코드 중복을 제거하고 기능을 확장합니다.



다형성 (Polymorphism)

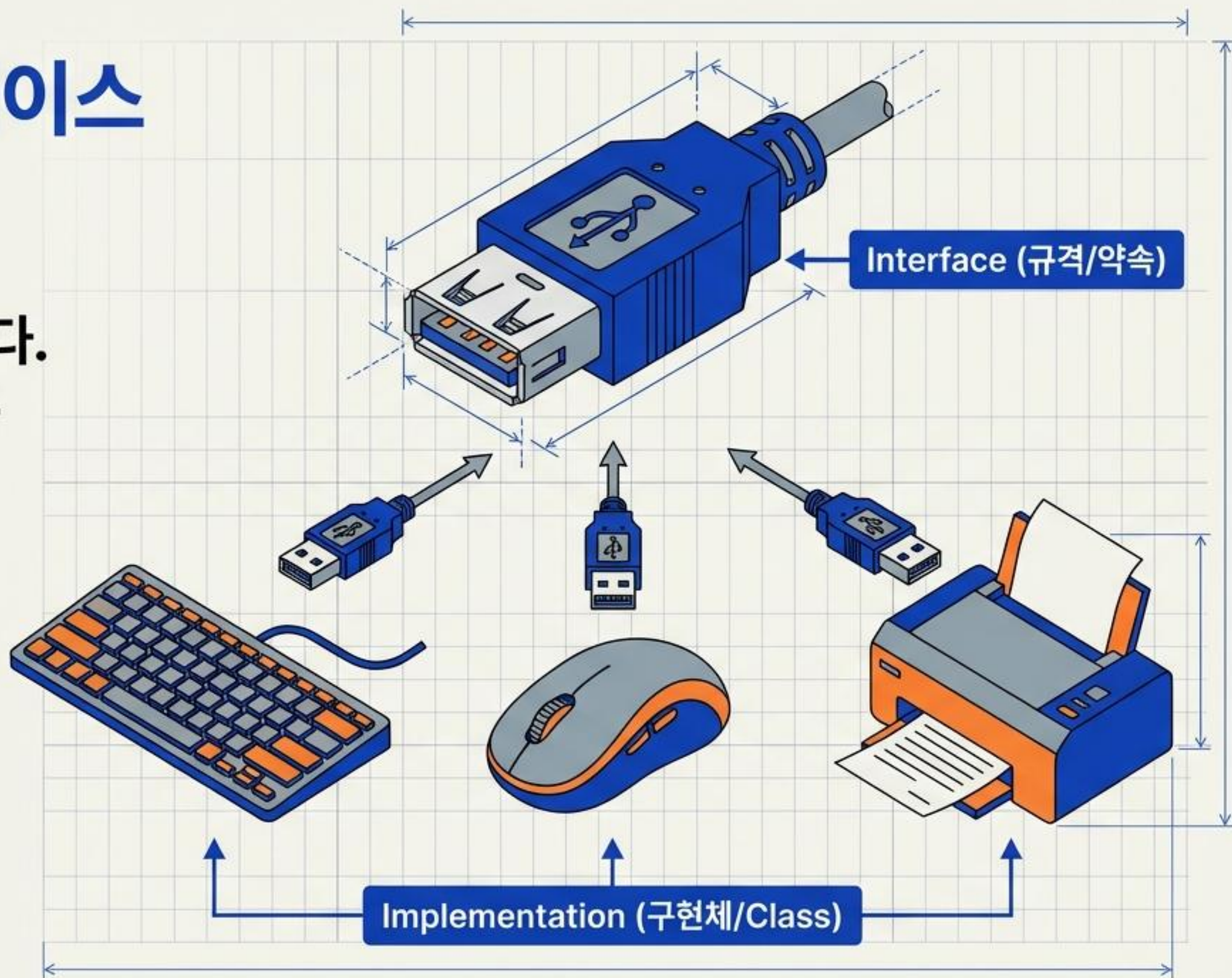


스프링(Spring)의 핵심 원리. 인터페이스 하나로 다양한 구현체를 교체하는 유연성.

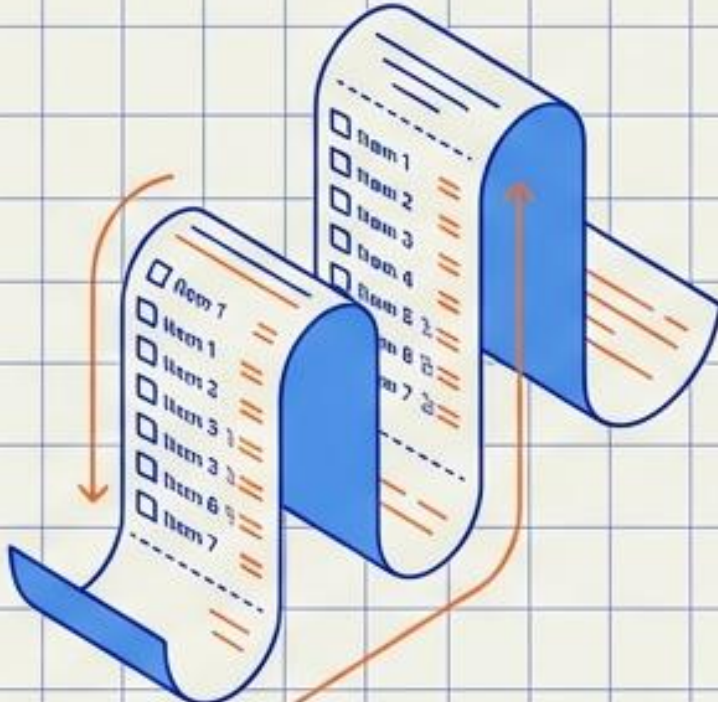
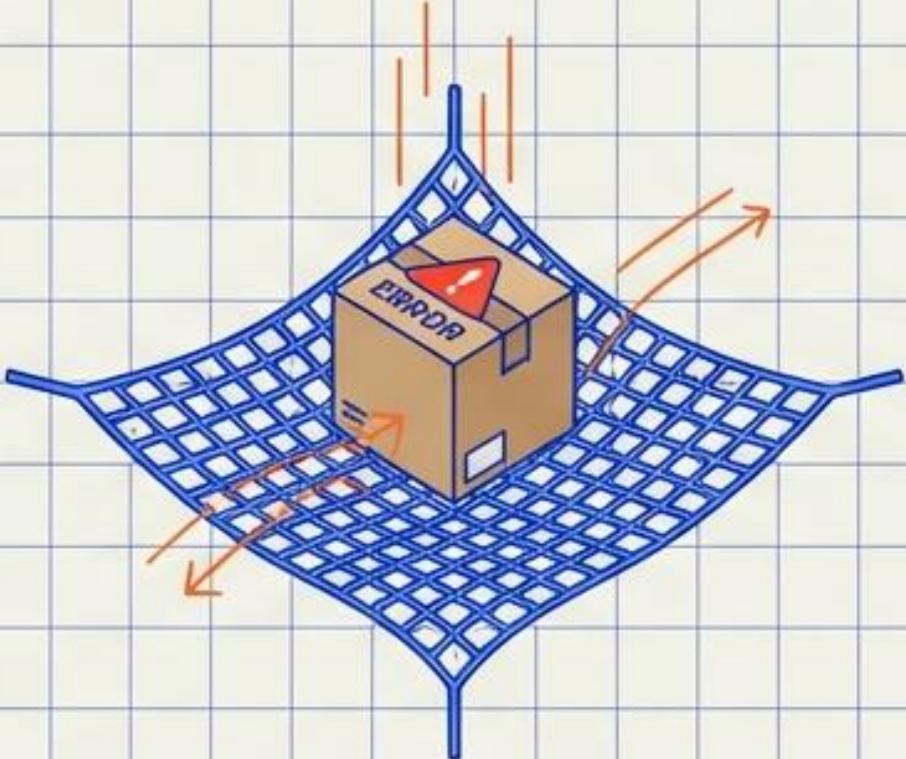
추상화 & 인터페이스 (Abstraction)

구현 방법은 몰라도 됩니다.
'규격'만 맞으면 무엇이든
연결할 수 있습니다.

Spring이 객체 간 결합도를 낮추는
핵심 방법.



The Survival Kit

List (ArrayList)	Map (HashMap)	Exception (예외처리)
		
순서가 있는 저장소 (중복 허용)	키(Key)와 값(Value)의 쌍 (JSON 구조)	서버가 죽지 않게 방어 (try-catch)

실무에서는 List로 담고, Map으로 찾고, Exception으로 방어합니다.

Modern Java: Stream & Lambda

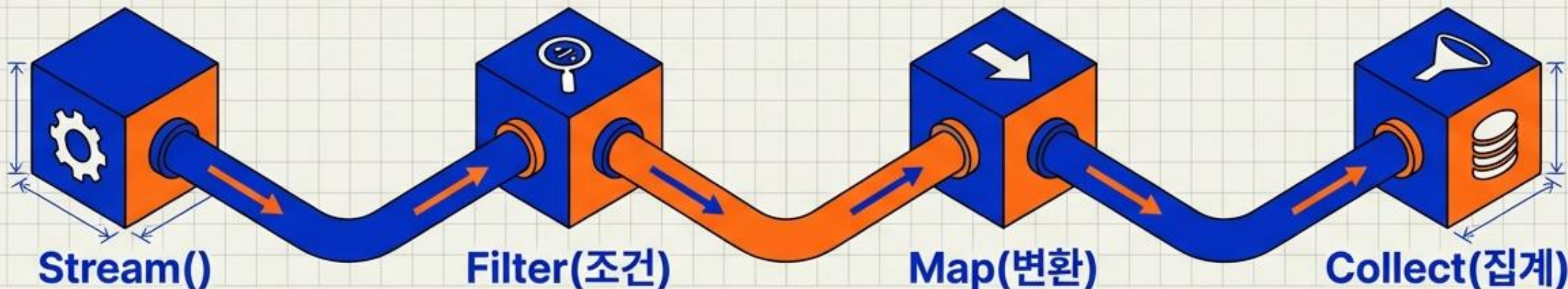
데이터를 흐름(Stream)으로 처리합니다. SQL처럼 직관적인 데이터 처리가 가능해집니다.

과거의 반복문

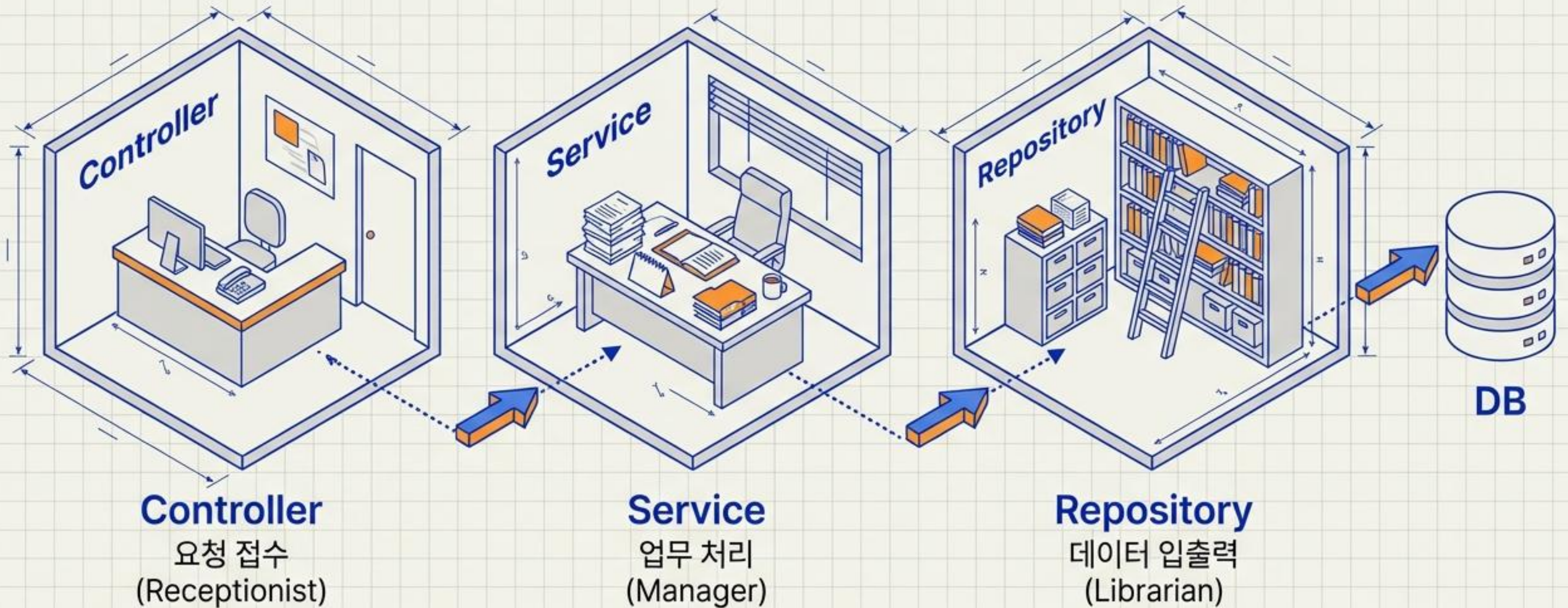
```
List<Data> results = new ArrayList<>();  
for (Data item : list) {  
    if (item.getValue() > 100) {  
        String transformed = transform(item.getName());  
        results.add(new Result(transformed));  
    }  
}  
// ... more boilerplate code ...
```



`list.stream().filter().map().collect()`



Spring MVC Pattern



객체들이 각자의 역할(Role)과 책임(Responsibility)을 나눠 협력합니다.

Portfolio Strategy: The Shopping Mall

단순히 코드를 짜는 것이 아니라, 데이터의 관계(Relationship)를 설계하는 것이 핵심입니다.



면접관이 반드시 묻는 객체지향 핵심 질문 (Core Interview Q&A)



Polymorphism (다형성)?

부모 타입으로 자식 객체를 제어하여,
코드 변경 없이 구현체를 교체하는 기술.



Interface vs Abstract?

인터페이스는 '약속(규격)'이고,
추상 클래스는 '미완성 설계도'입니다.



Encapsulation (캡슐화)?

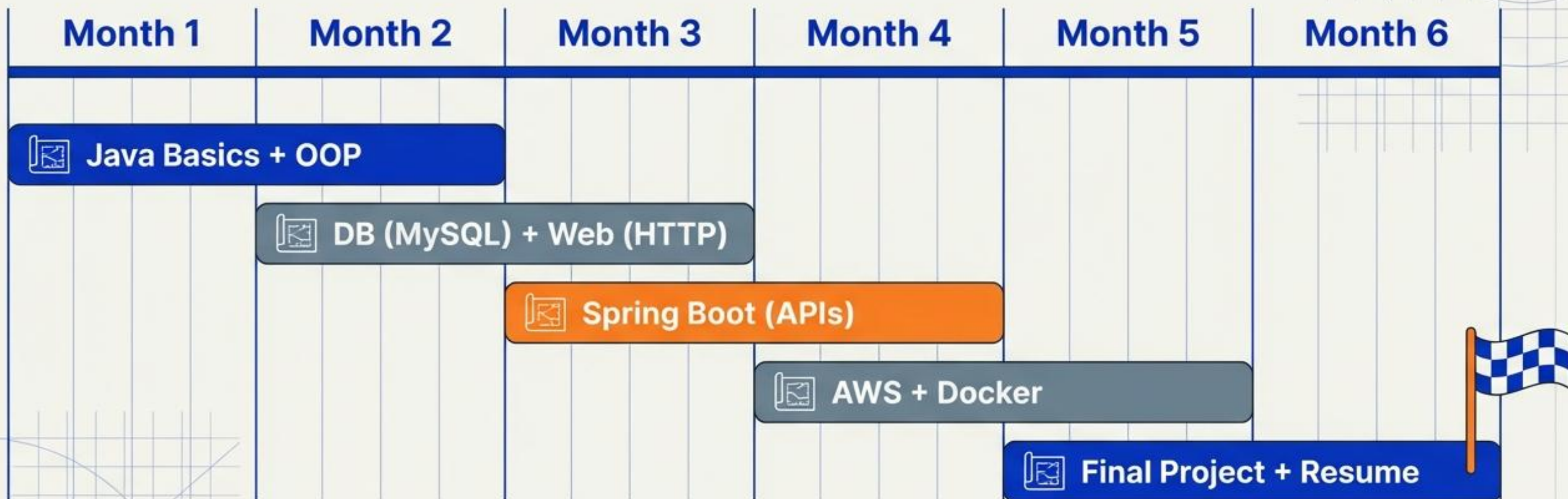
데이터를 보호하고, 정해진 메서드를
통해서만 접근하게 하여 무결성을 지킵니다.



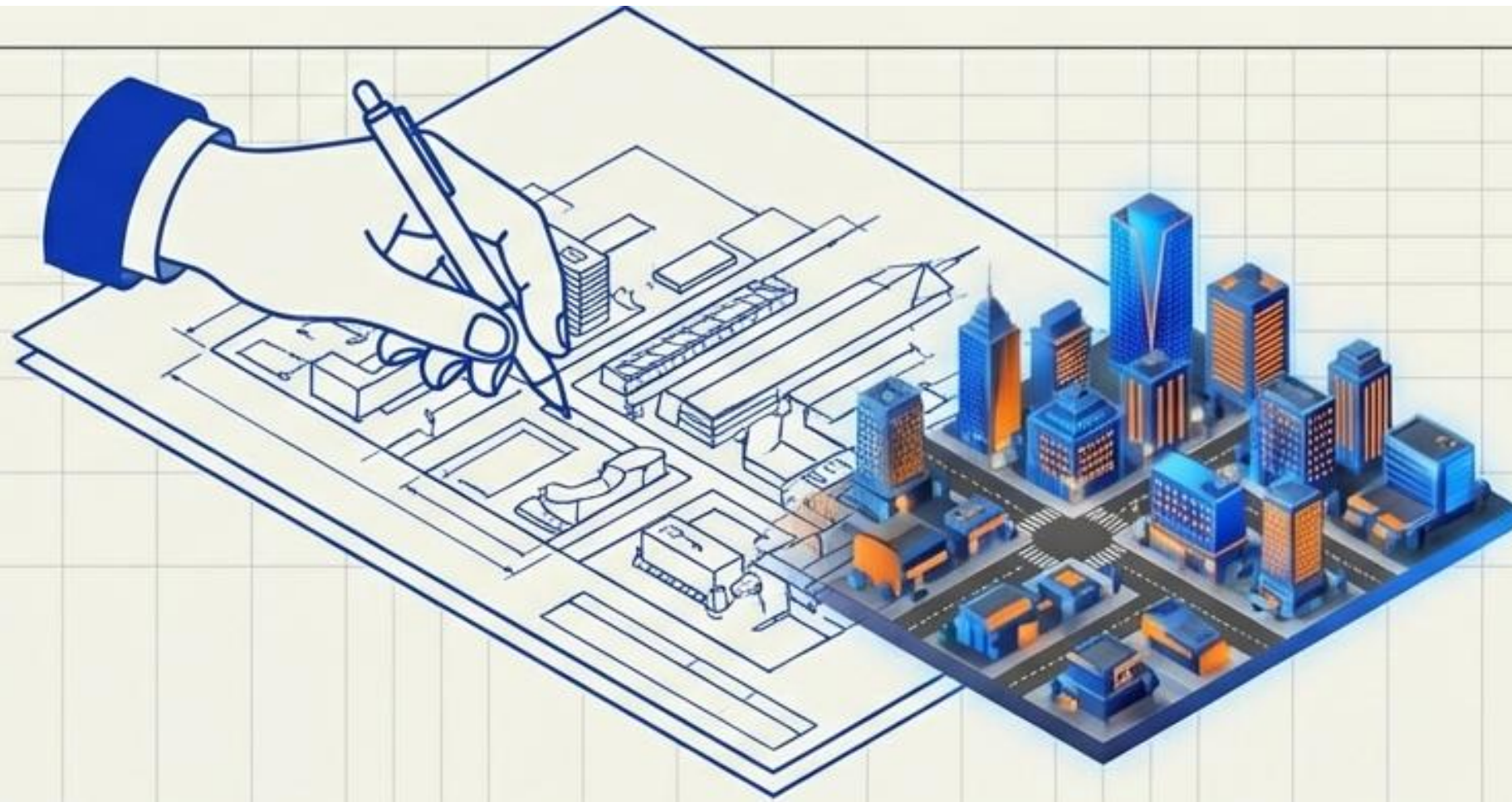
Memory Areas?

Static (설계도) / Stack (메서드 실행) /
Heap (실제 객체).

6-Month Roadmap (Non-Major to Hired)



10개의 튜토리얼보다, 완성된 1개의 프로젝트를
끝까지 배포해보는 것이 취업의 지름길입니다.

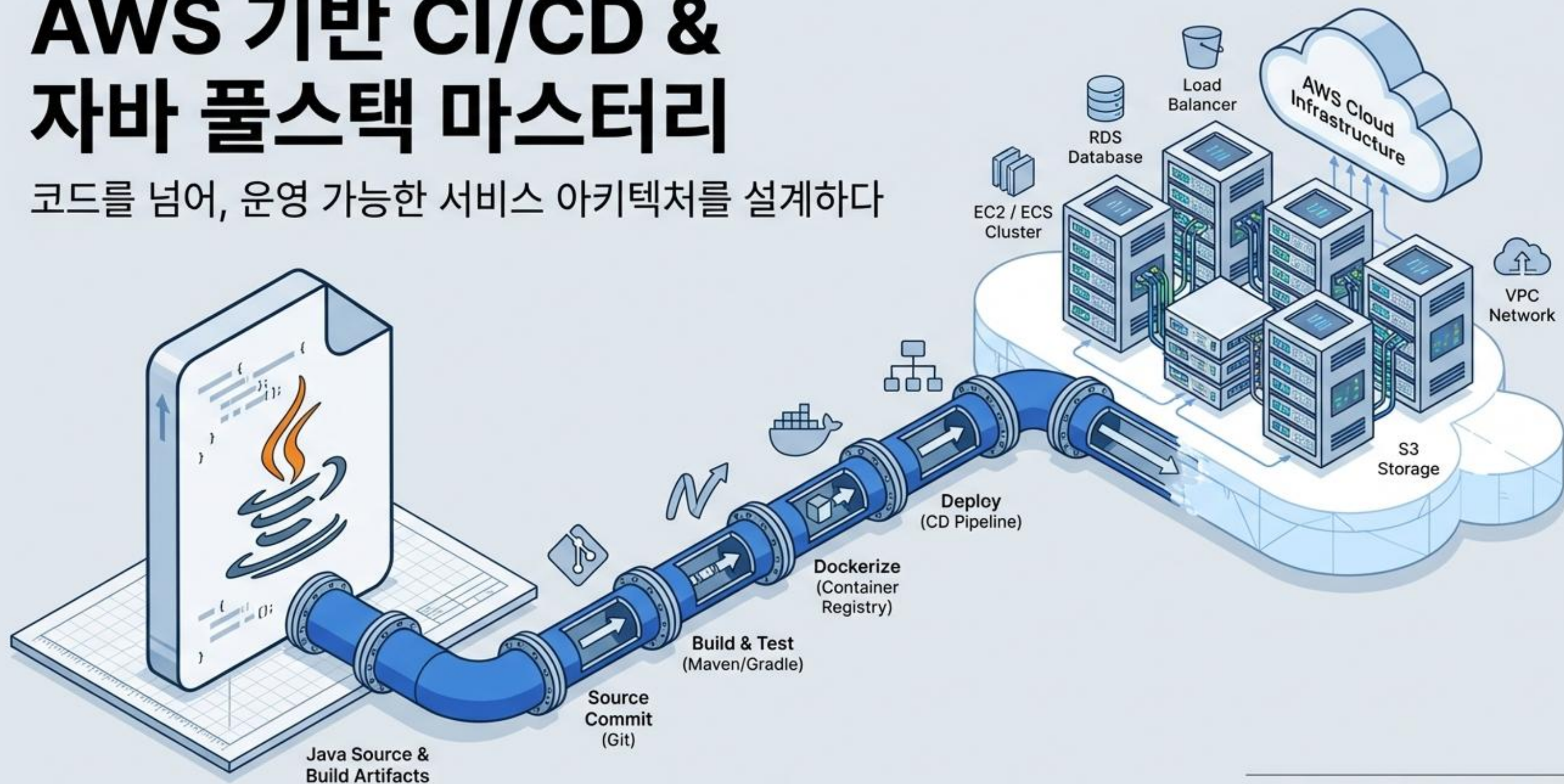


자바는 단순한 문법 암기가 아닙니다.
세상을 설계하는 방법입니다.

Start Coding Your Reality 

AWS 기반 CI/CD & 자바 풀스택 마스터리

코드를 넘어, 운영 가능한 서비스 아키텍처를 설계하다



개발을 넘어 운영까지 (Beyond Development to Operation)

Frontend



화면

(View)

Noto Sans KR
Inter

Backend



서버

(Server Logic)

Noto Sans KR
Inter

Database



데이터

(Data Persistence)

Noto Sans KR
Inter

Cloud



서버 운영

(Infrastructure)

Noto Sans KR
Inter

DevOps



자동 배포

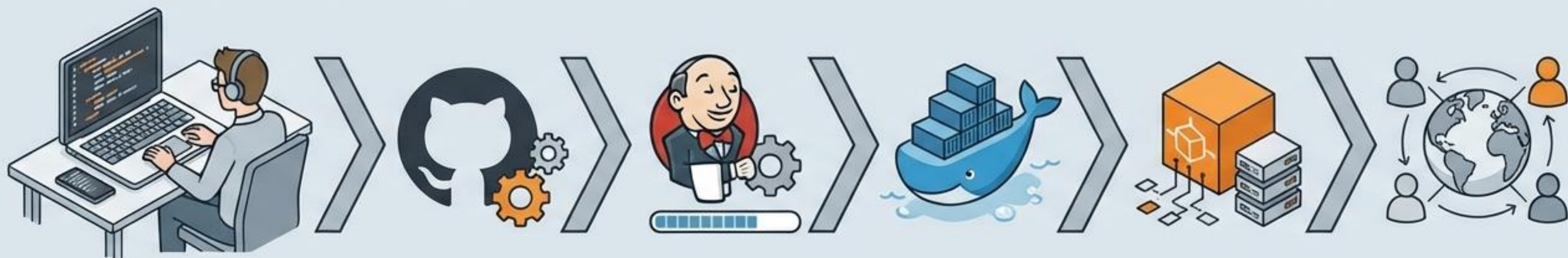
(Automation)

Noto Sans KR
Inter

코드를 작성하는 것과 서비스를 운영하는 것은 다릅니다.

서비스의 전체 흐름 (The Complete Service Flow)

코드를 만들고 → 자동으로 배포되고 → 운영되는 과정



1. 개발자
(코드 작성)
Developer
(Writing Code)

2. GitHub
(업로드)
JetBrains Mono
(Upload)

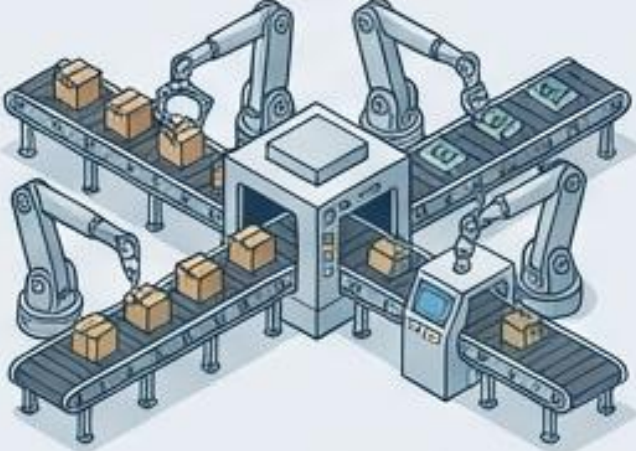
3. Jenkins
(자동 빌드)
Jenkins
(Automated Build)

4. Docker
(이미지 생성)
Docker (Image
Creation)

5. AWS EC2
(서버 배포)
AWS EC2
(Server
Deployment)

6. User
(서비스 접속)
User (Service
Access)

복잡한 개념, 쉽게 이해하기 (Infrastructure Analogies)

	Technology		Real World Analogy
 AWS	 건물 임대 (서버를 사지 않고 빌리는 것)	 Jenkins	 자동화 직원 (배포를 대신해주는 로봇)
 Docker	 이사 박스 (환경을 통째로 포장)  창고 (코드 저장소)	CI/CD	 자동 공장 (무인 배포 시스템)

강력한 백엔드 표준, Java & Spring Boot

REST API

Frontend와
데이터 통신



JWT

보안 로그인 및
인증 (Auth)



JPA/DB

데이터베이스 연동
및 SQL 관리

금융, 대기업, SI의 표준 기술.

단순한 코딩이 아닌 '웹 서버 프레임워크'를 다룹니다.

어디서든 동일하게 실행되는 서버, Docker

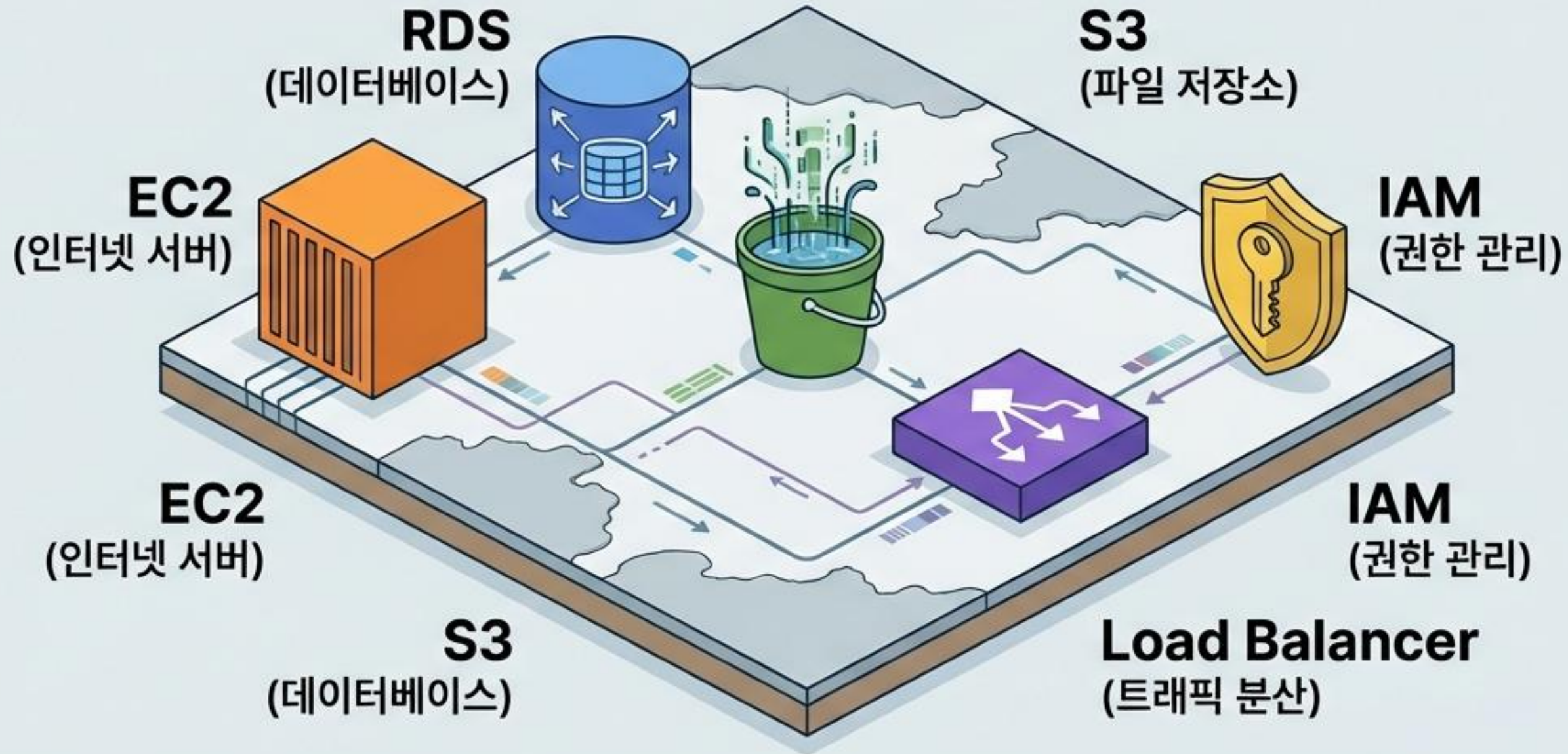


```
FROM openjdk:17  
COPY app.jar app.jar  
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

내 PC에서는 되는데
서버에서는 안 됨?

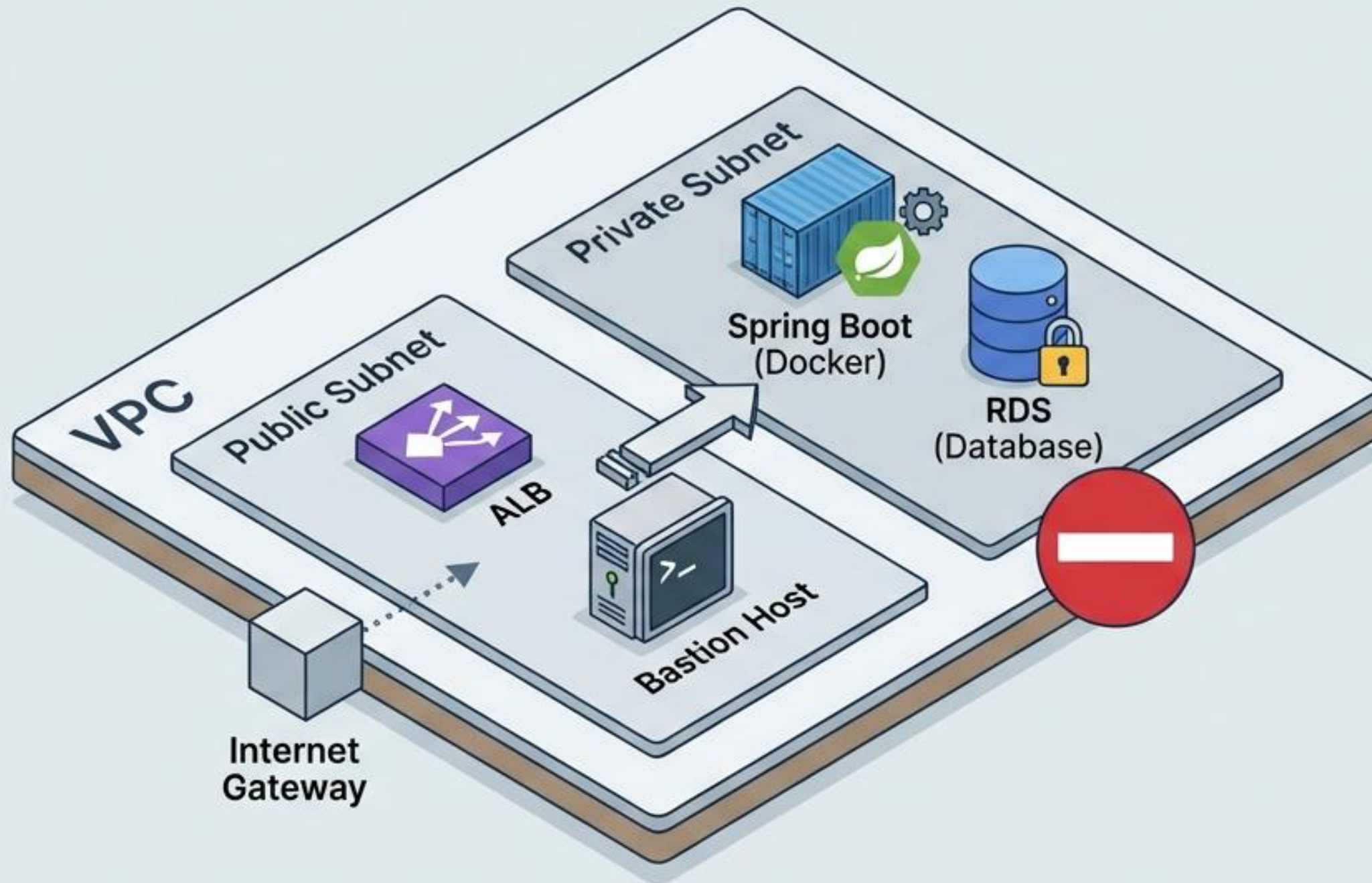
서버 환경을 통째로 포장하여 배포
(이사 박스처럼, 환경을 그대로 옮깁니다).

인터넷 위의 내 서버, Amazon Web Services



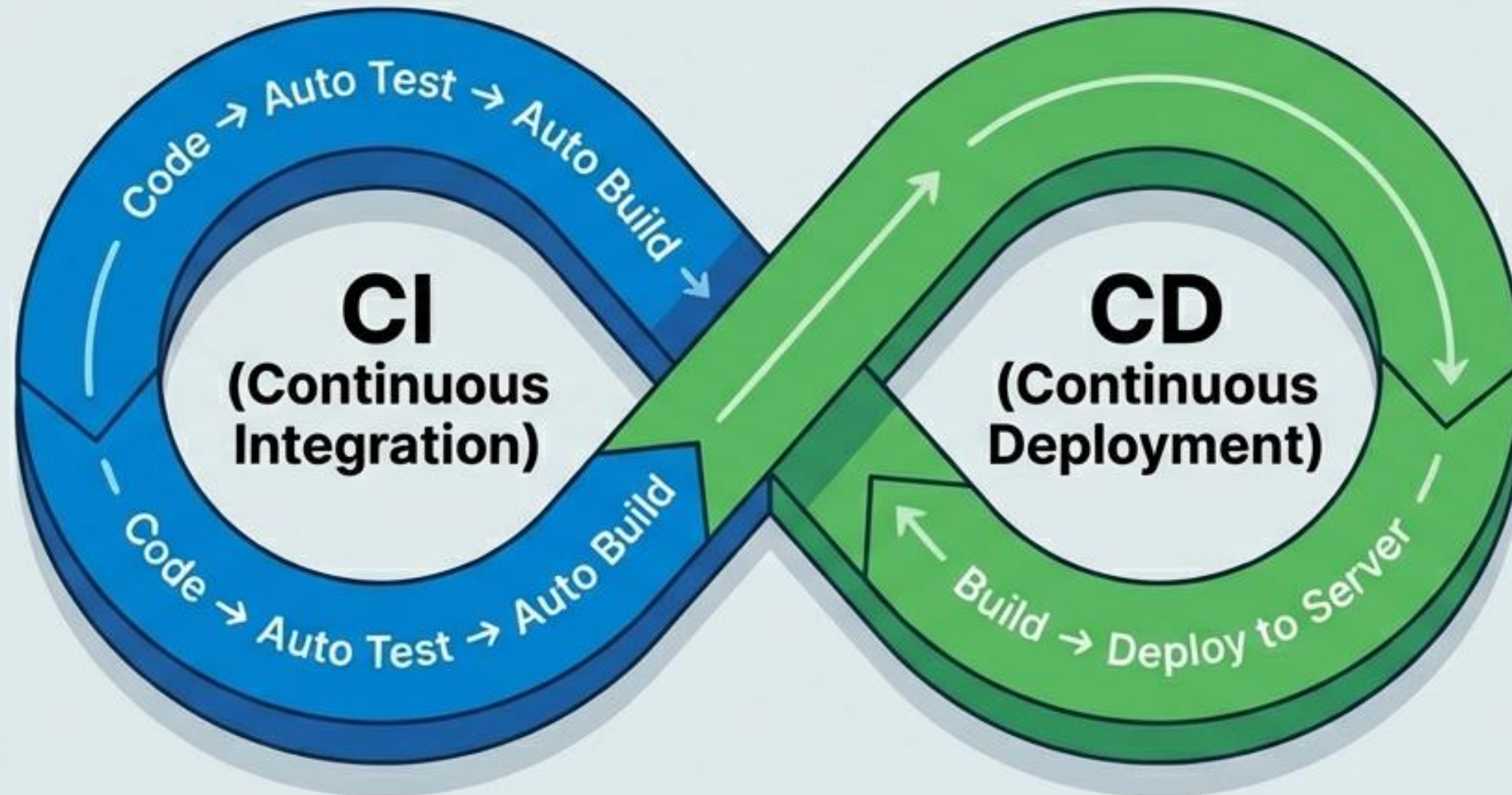
컴퓨터를 직접 사지 않고, 인터넷에서 필요한 만큼 빌려 쓰는 인프라.

안전한 인프라 설계 (VPC Network Architecture)



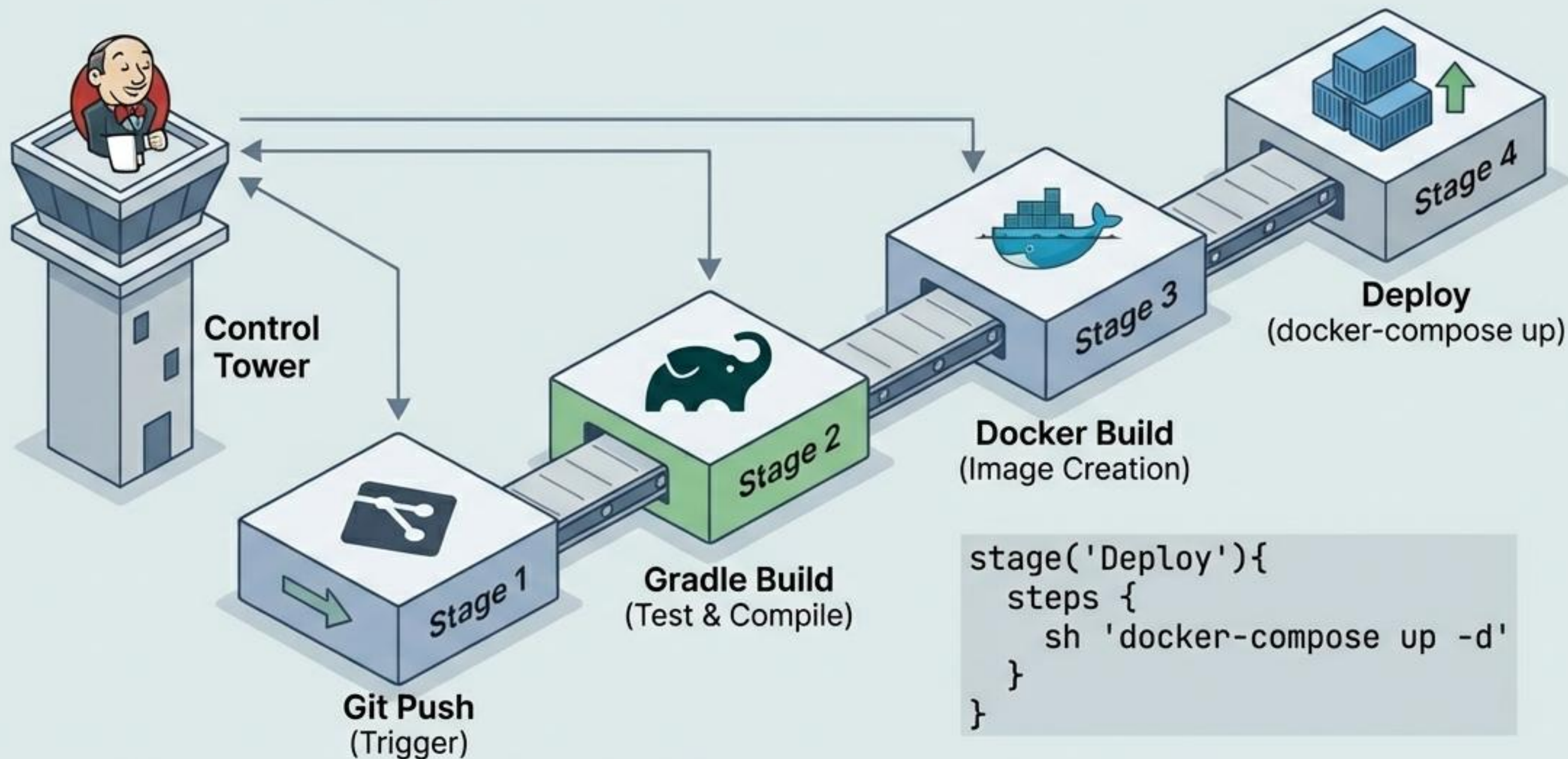
- DB는 절대 Public에 두지 않는다 (Private Subnet 배치).
- 외부 접속은 Load Balancer와 Bastion Host를 통해서만 허용.
- SSH 접속:
`ssh -i key.pem ubuntu@IP`

자동화의 마법, CI/CD



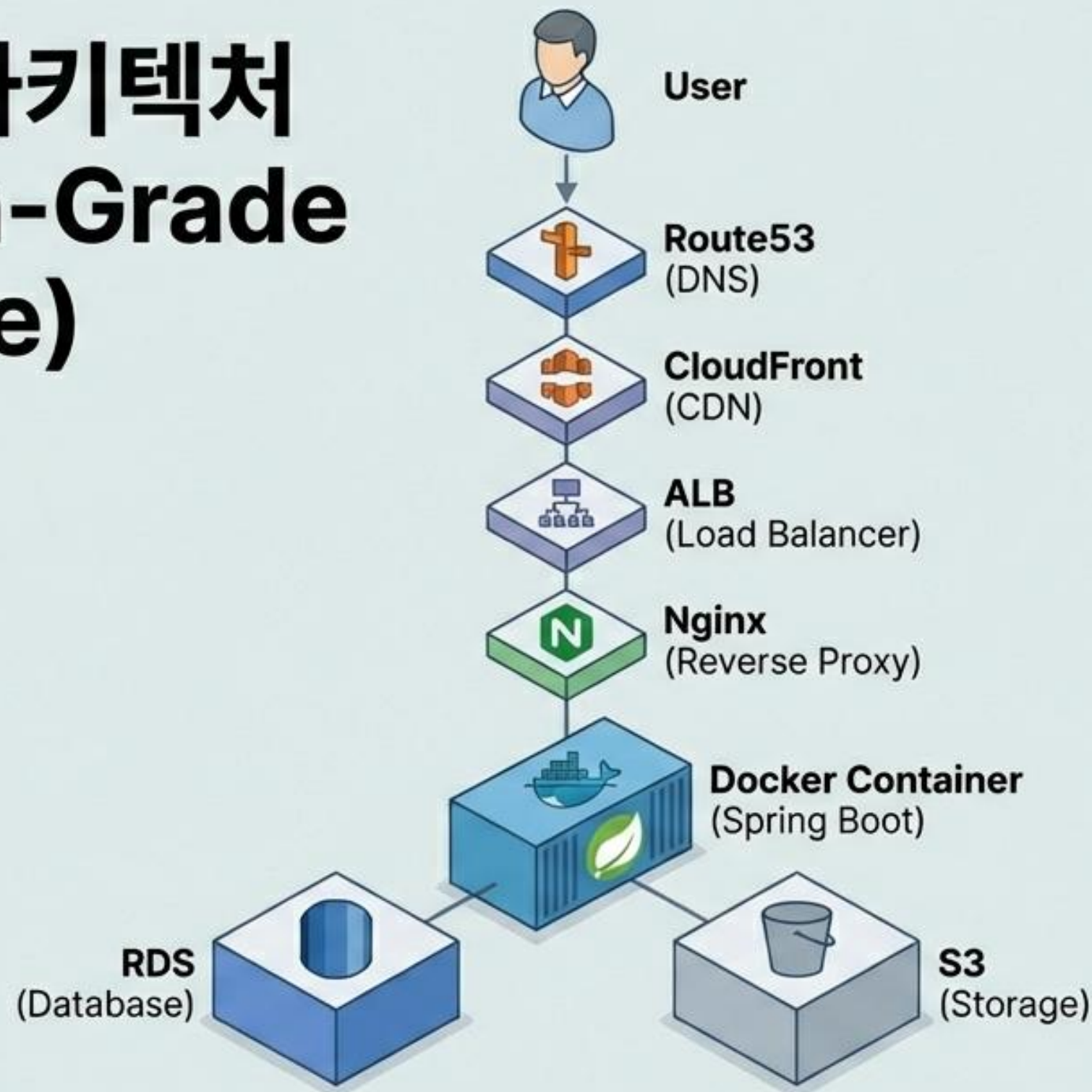
개발자가 Git Push만 하면 끝. 사람이 서버에 접속하지 않음.

자동 배포 명령탑, Jenkins



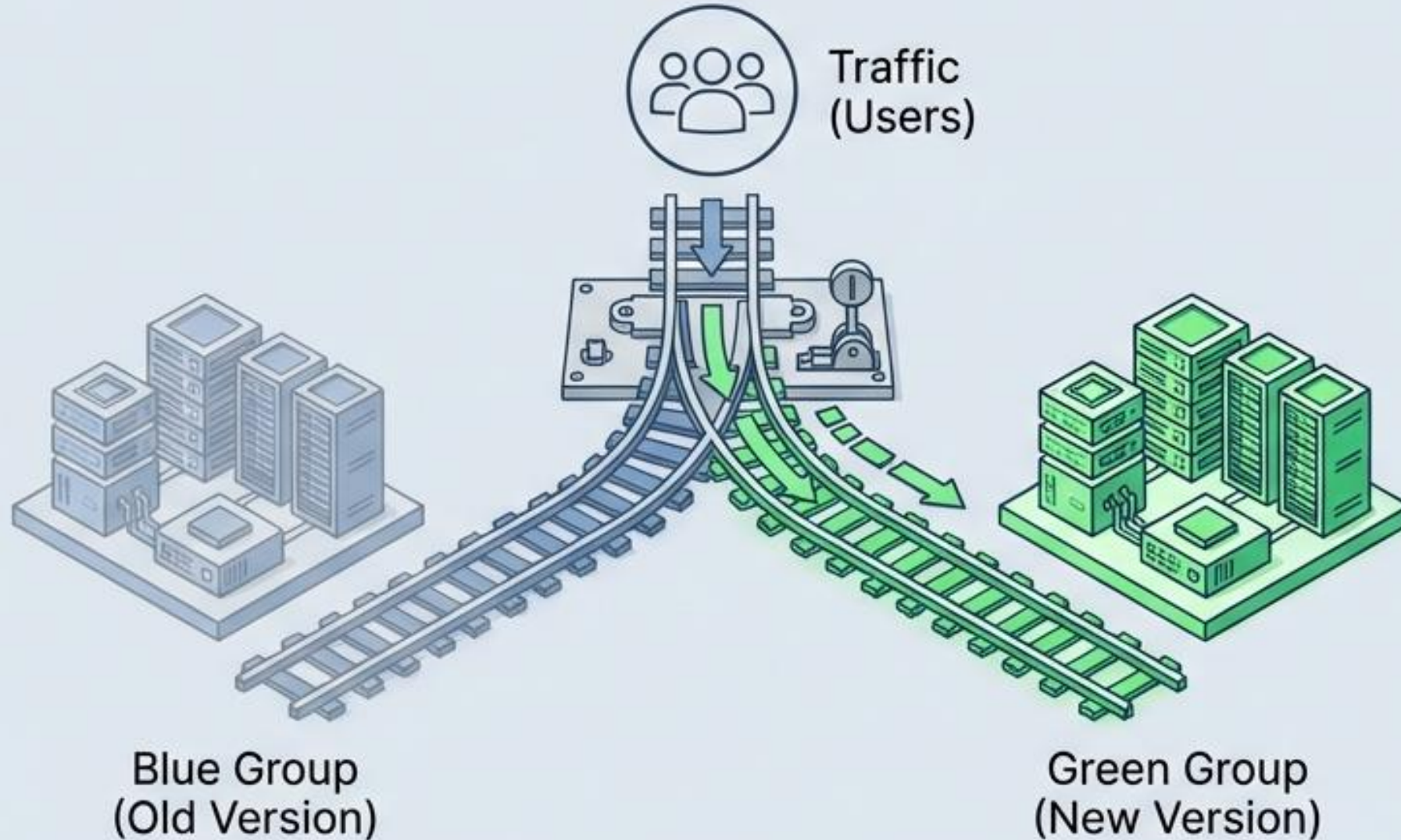
GitHub Webhook이 코드를 감지하고 Jenkins가 배포를 수행합니다.

실무 수준의 아키텍처 (Production-Grade Architecture)



신입 백엔드/DevOps 포트폴리오 상위 10%에 해당하는 구조.

멈추지 않는 서비스, 무중단 배포 (Blue/Green)



1. Green 버전(신규) 배포 및 실행.
2. Health Check 확인.
3. Nginx/ALB가 트래픽을 Green으로 전환.
4. Blue 버전 종료.

배포 중에도 사용자는 서비스를 계속 이용할 수 있습니다.

초보자가 피해야 할 치명적 실수 (Common Pitfalls)



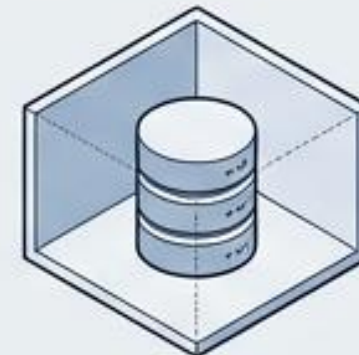
Root 계정 사용

반드시 IAM User와 권한(Role)을 생성하여 사용.



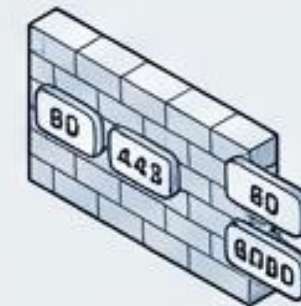
DB Public 오픈

DB는 해킹 대상 1순위. Private Subnet에 격리.



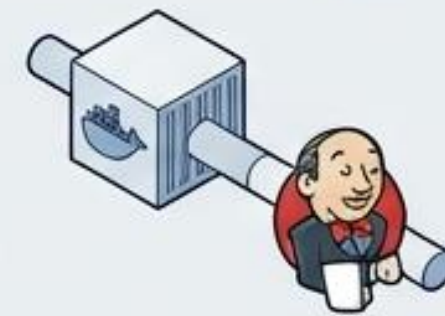
Security Group 전체 허용

필요한 포트(80, 443, 8080)만 정확히 개방.

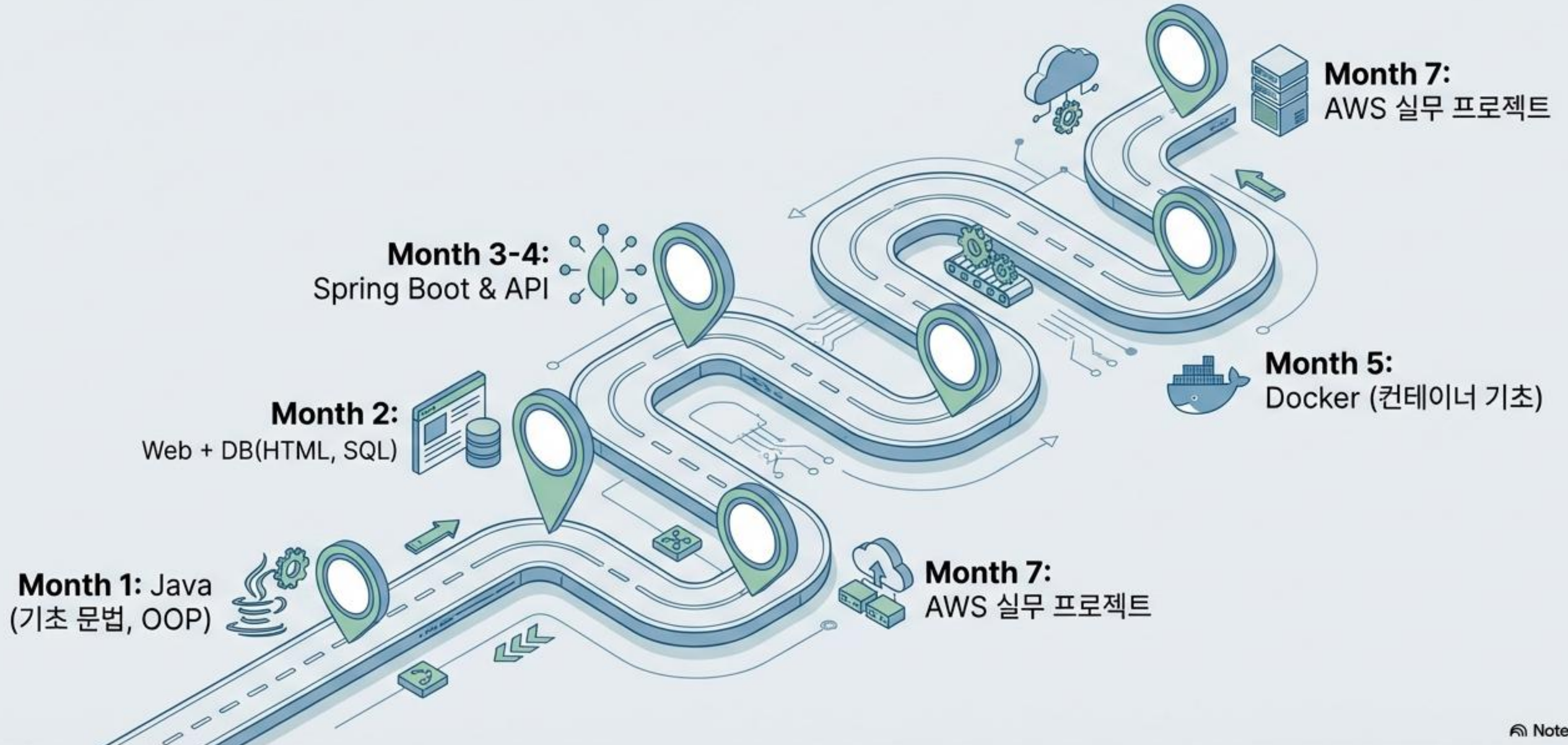


수동 배포 (FTP)

FTP 사용 금지. Docker와 Jenkins로 자동화.

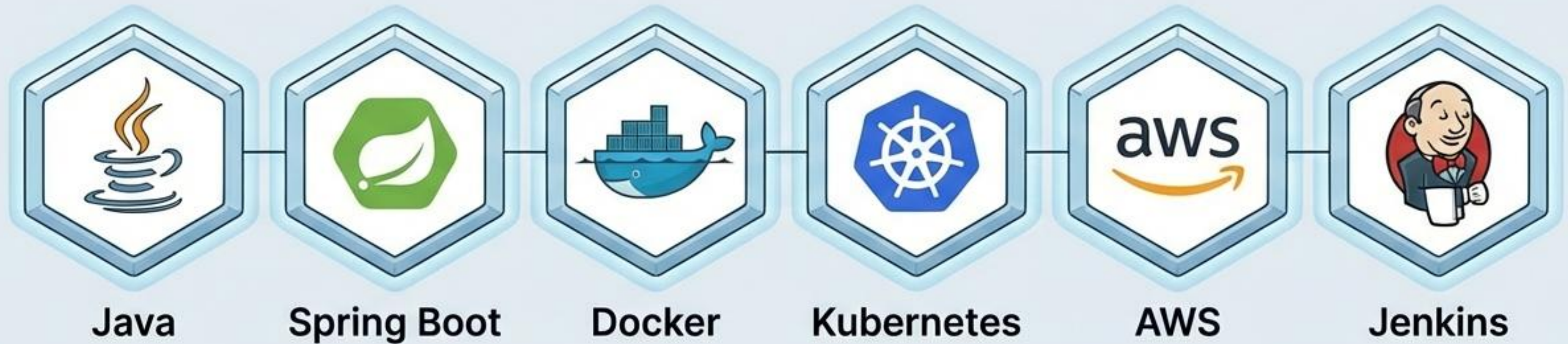


7개월의 여정 (The Curriculum Roadmap)

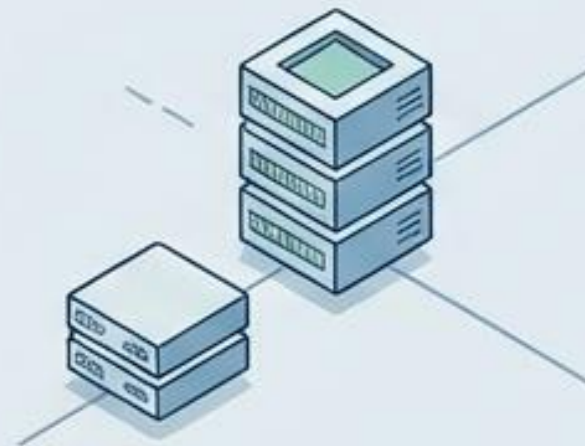


당신이 갖게 될 강력한 무기

단순한 코더가 아닌, 서비스 아키텍트(Service Architect)로 성장합니다.

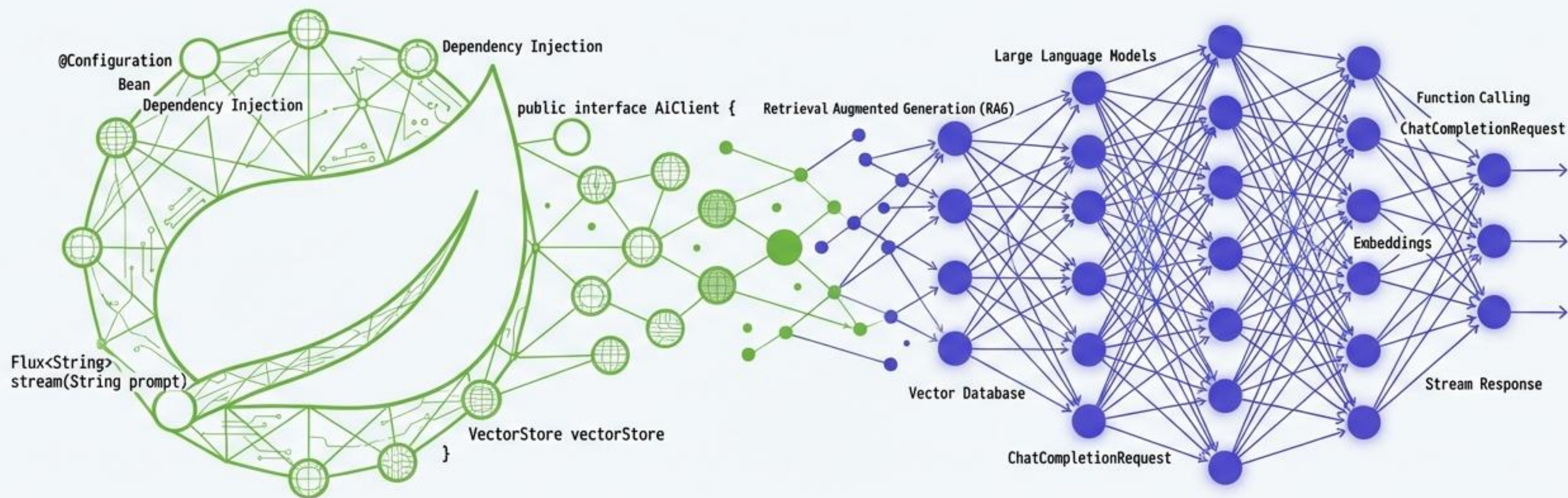


- ✓ 로그인 웹사이트
- ✓ REST API 서버
- ✓ AWS 인프라 구축
- ✓ CI/CD 자동 배포
- ✓ 무중단 운영 능력



Spring AI: 자바 개발자를 위한 오픈소스 AI 풀스택

생성형 AI 기초부터 RAG, 함수 호출까지—스프링 생태계로 완성하는 AI 엔지니어링



생성형 AI와 모델의 진화



LLM (Large Language Model)

텍스트 중심 (Text-centric)

JetBrains Mono



LMM (Large Multimodal Model)

텍스트 + 이미지 + 오디오 + 비디오 통합 처리

JetBrains Mono

생성형 AI (Generative AI): 기존 데이터 패턴 학습 → 새로운 콘텐츠 창조

단순 분석을 넘어, 인간의 인지 기능을 모방하여 창의적 문제 해결 자동화.

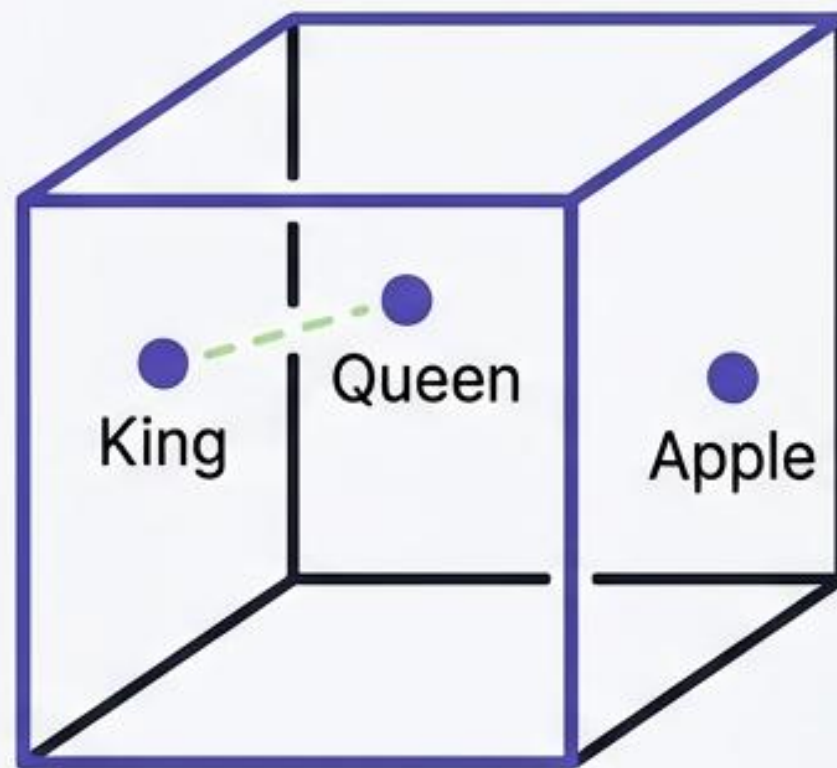
토큰 (Tokens)



Token = Money (Cost)

- AI 모델의 기본 처리 단위.
입력/출력 모두 비용 발생.
- Context Window: 한 번의 호출로
처리 가능한 텍스트 한계

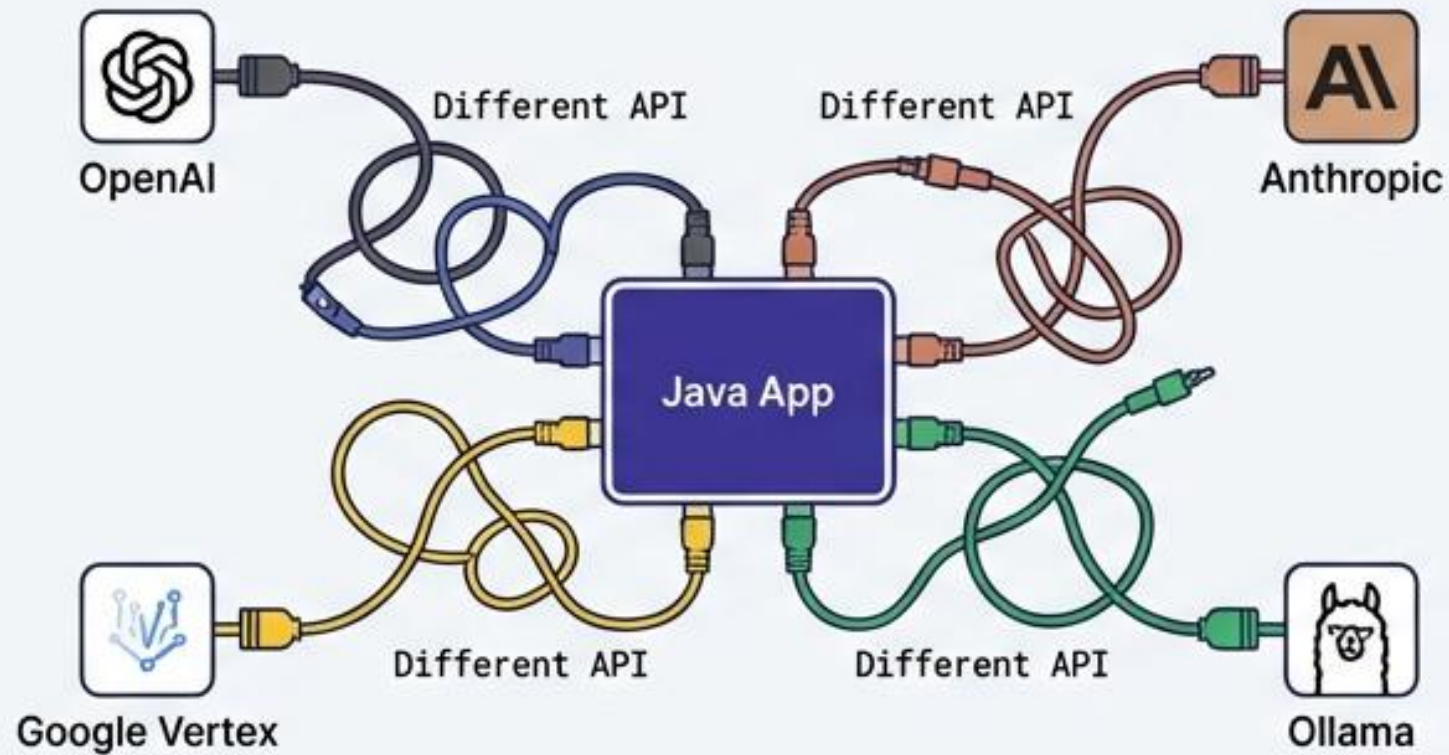
임베딩 (Embeddings) & 벡터 (Vectors)



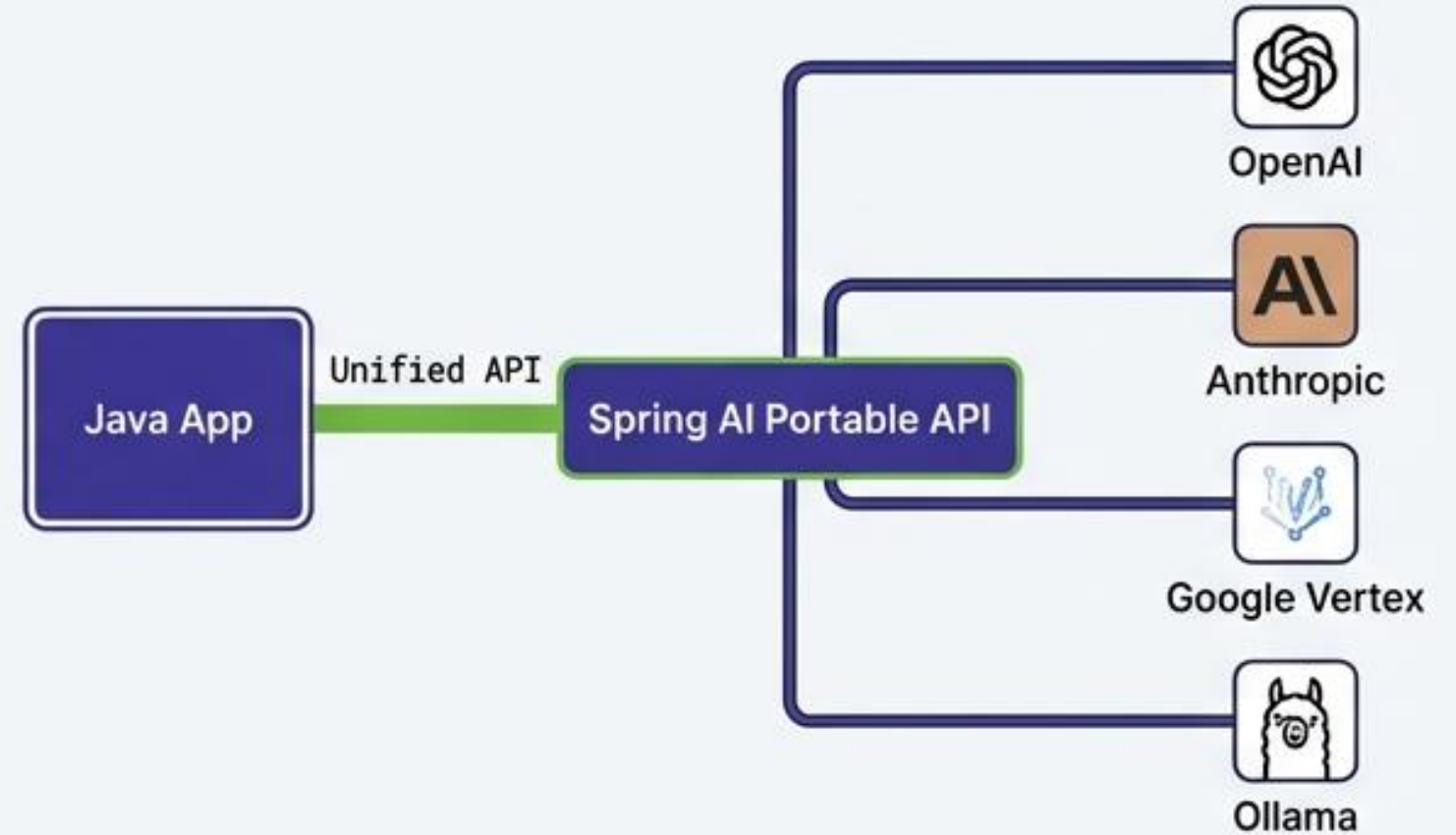
- 텍스트를 벡터(숫자 배열)로 변환.
- 의미 공간(Semantic Space)에서 점들의
거리가 의미의 유사성을 결정합니다.

파편화된 AI 생태계와 Spring AI의 해법

Before



After

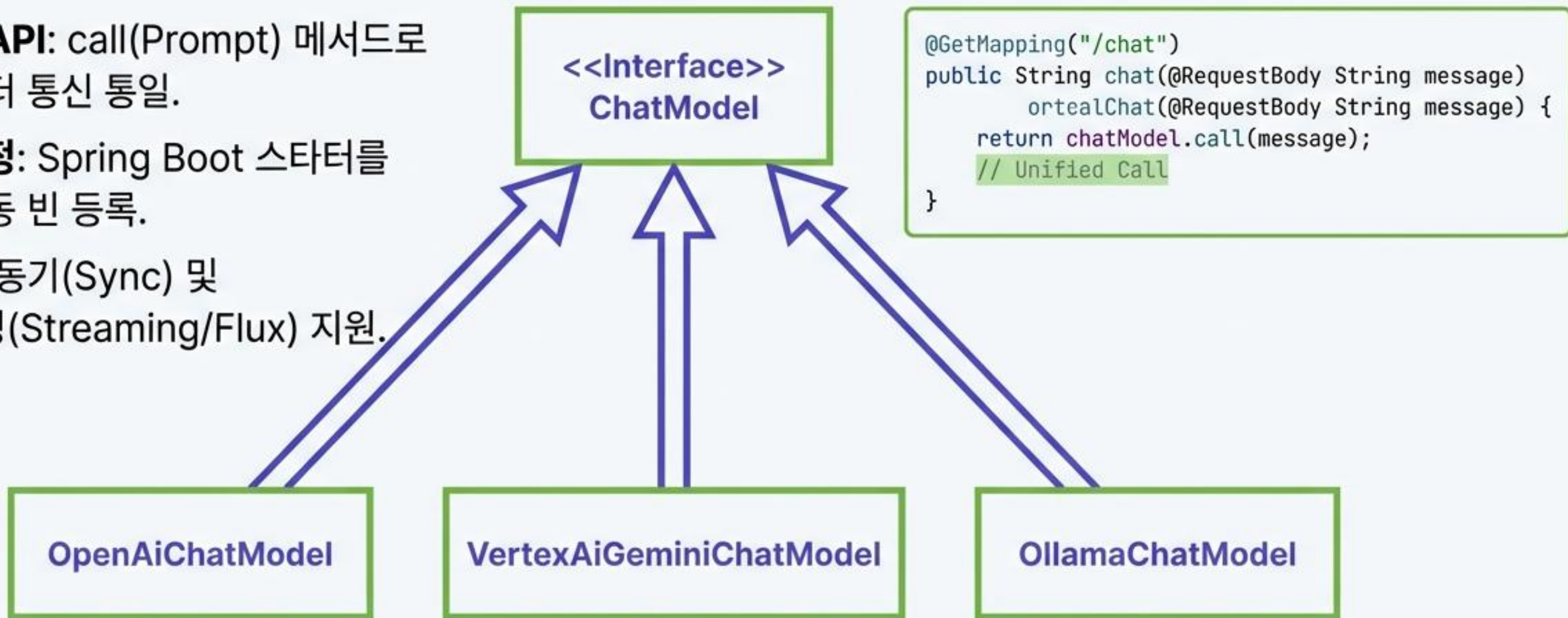


- **이식성 (Portability):** 코드 변경 없이 AI 모델(벤더) 교체 가능.
- **모듈화 (Modularity):** POJO를 사용한 유연한 설계.

OpenAI의 GPT에서 Anthropic의 Claude로 교체 시, 비즈니스 로직 수정 불필요.

핵심 추상화 계층: ChatModel

- **일관된 API:** call(Prompt) 메서드로 모든 벤더 통신 통일.
- **자동 설정:** Spring Boot 스타터를 통한 자동 빈 등록.
- **유연성:** 동기(Sync) 및 스트리밍(Streaming/Flux) 지원.



프롬프트 엔지니어링: PromptTemplate

Spring MVC View Template $\approx$ Spring AI Prompt Template



```
PromptTemplate template = new PromptTemplate(command);
template.add("topic", "life");
Message userMessage = template.render();
```

System Message: 모델의 페르소나 정의
User Message: 사용자의 질문

고수준 API: ChatClient

```
return chatClient.prompt()
```

```
    .system("You are a friendly pirate.")
```

```
    .user("Tell me a story about Spring.")
```

```
    .options(ChatOptions.builder()
```

```
        .temperature(0.7)
```

```
        .build())
```

```
    .call()
```

```
    .content();
```

← System Persona 설정

← User Input 입력

← 모델 호출 실행

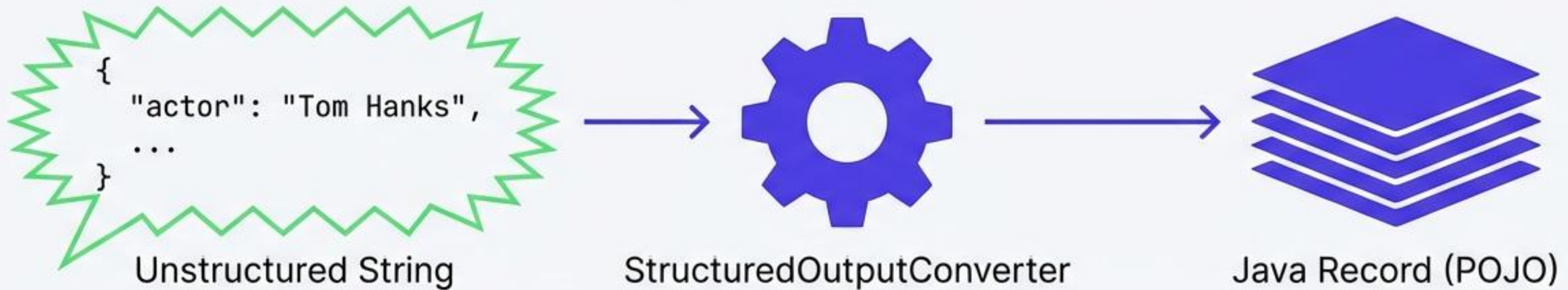
Fluent API

메서드 체이닝으로
가독성 향상.

Default Settings

@Configuration에서
기본 메시지 설정 가능.

구조화된 출력: String에서 POJO로



```
1 public record ActorFilms(String actor, List<String> movies) {}  
2  
3 ActorFilms films = chatClient.prompt()  
4     .user("Generate filmography for Tom Hanks")  
5     .call()  
6     .entity(ActorFilms.class); // Automatic Mapping
```

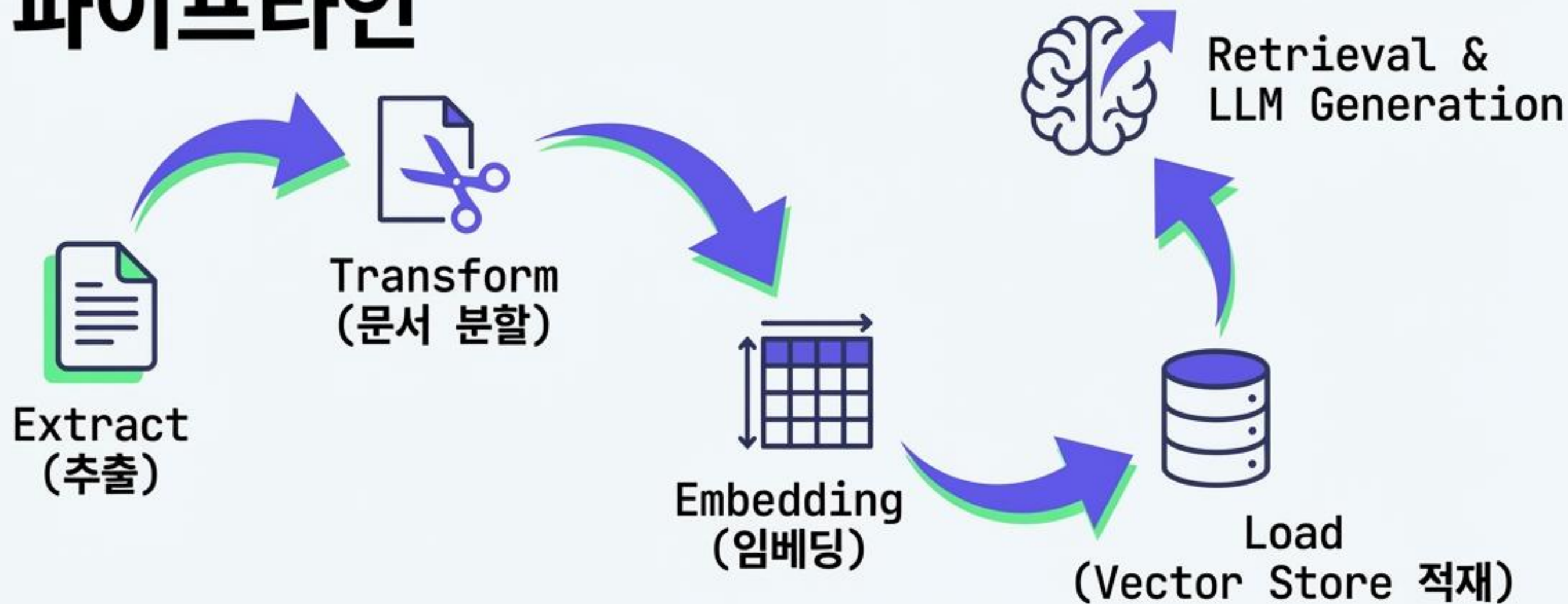
Java `record` (불변 객체)와 완벽한 호환.

모델 제어와 설정: ChatOptions



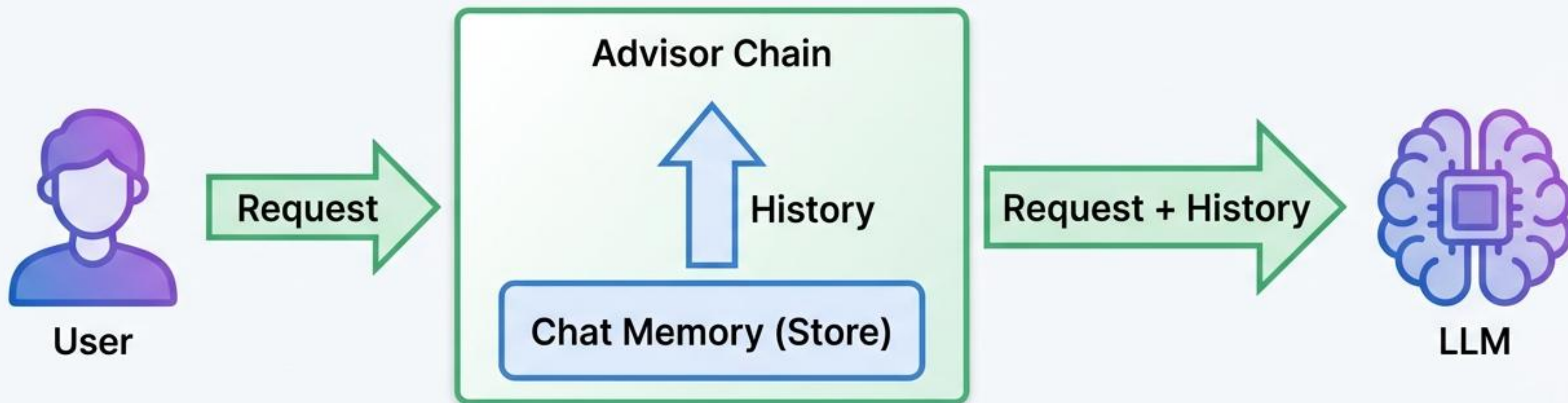
Portability: 벤더별 설정(예: OpenAI `stop` vs Anthropic `stop\_sequences`)을 Spring AI가 자동 변환합니다.

검색 증강 생성 (RAG) & ETL 파이프라인



Supported Stores: Redis, PostgreSQL (PGVector), MongoDB Atlas, Pinecone

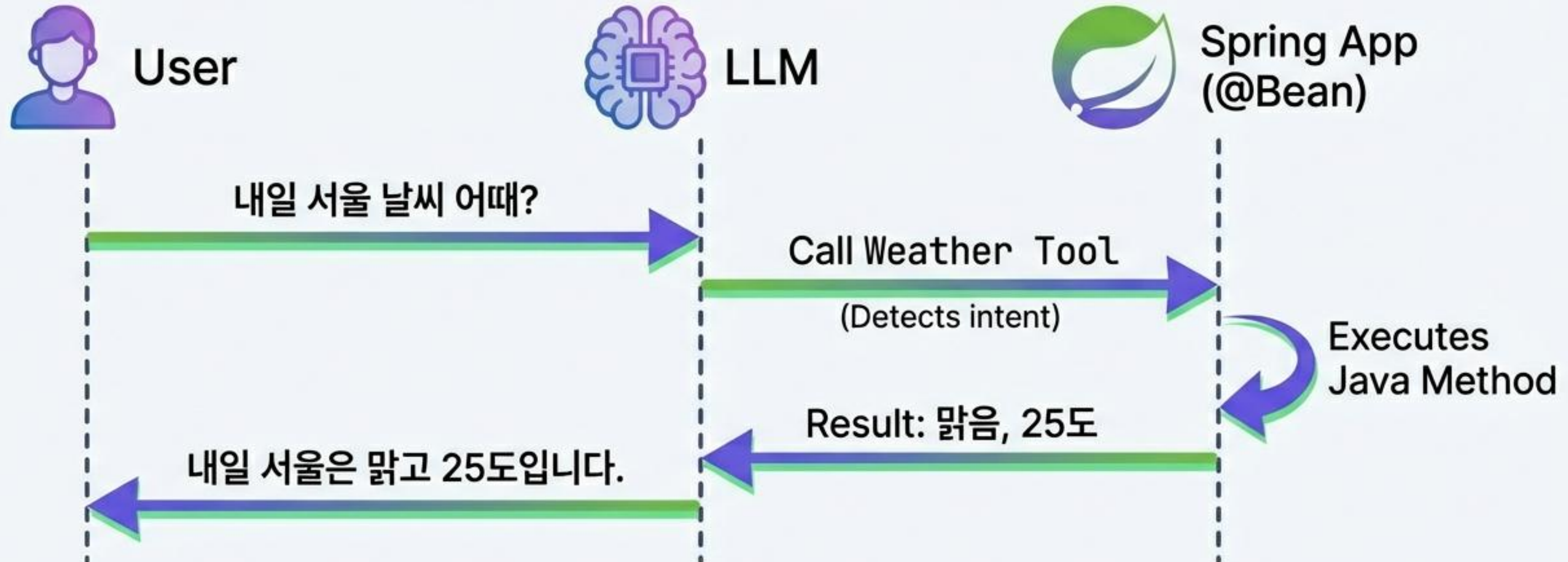
챗 메모리: 대화 맥락 유지



문제: LLM은 상태 비저장(Stateless). 이전 대화를 기억하지 못함.

해결: `MessageWindowChatMemory` 등으로 대화 버퍼 관리.

함수 호출 (Function Calling / Tool Calling)



`java.util.function.Function`을 `@Bean`으로 등록하여 AI 도구로 사용.

로컬 AI와 Ollama 연동



OpenAI / Cloud

- 비용 발생, 데이터 전송

Spring AI

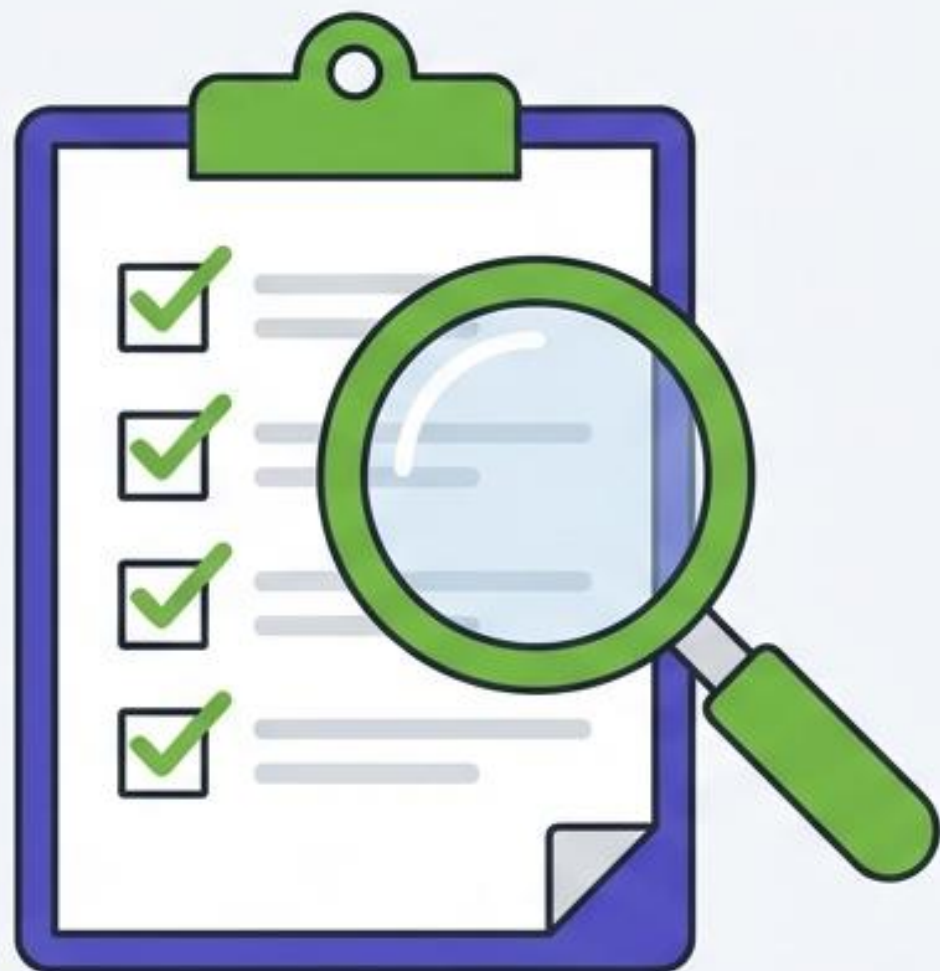


Ollama / Local

- 비용 0원, 데이터 보안, 오프라인

- **비용 절감** : 토큰 비용 무료.
- **보안** : 사내망/폐쇄망 환경에서 데이터 유출 방지.
- **호환성** : OllamaChatModel을 통해 OpenAI와 동일한 API 사용.

AI 응답 평가 및 테스트 (Evaluation)



관련성 (Relevance)

질문에 맞는 답변인가?



일관성 (Consistency)

논리적 모순은 없는가?



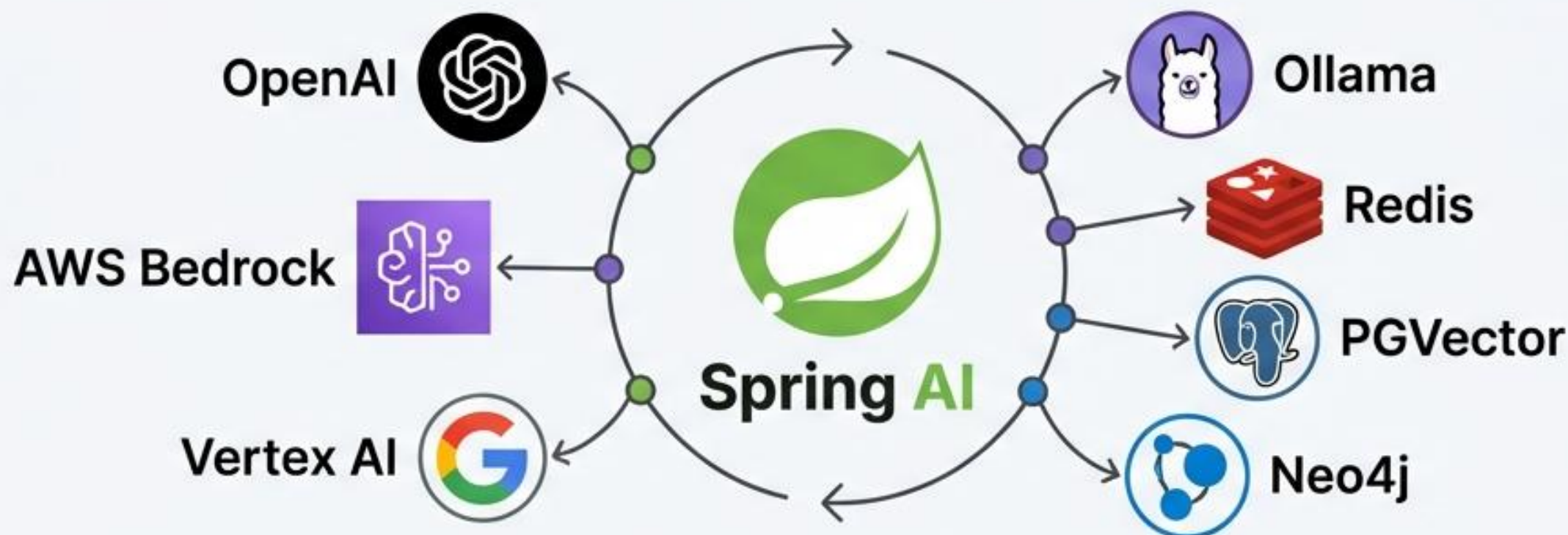
사실적 정확성 (Accuracy)

할루시네이션(거짓 정보) 없음.



Evaluator: 다른 AI 모델을 사용하여
응답을 검증 (Model-graded evaluation).
JUnit 테스트 통합 지원.

Spring AI: 자바 생태계의 AI 표준



Familiarity

Spring Boot + POJO



Portability

코드 수정 없는 벤더 교체



Productivity

RAG, Memory, Function
Calling의 추상화

`'spring-ai-starter'`로 지금 바로 AI 엔지니어링을 시작하세요.